



TIỂU LUẬN CUỐI KỲ

MÔN HỌC: XỬ LÝ ẢNH SỐ

HỆ THỐNG NHẬN DIỆN KÝ HIỆU TAY ĐỂ ĐIỀU KHIỂN THIẾT BỊ IOT

Giảng viên: PGS.TS. Hoàng Văn Dũng

Danh sách sinh viên thực hiện

Mã số SV	Họ và tên	Mức độ đóng góp (%)
23110203	Phạm Trần Thiên Đăng	100
23110235	Phạm Võ Nhất Kha	100
23110280	Huỳnh Thanh Nhân	100
23110327	Huỳnh Ngọc Thắng	100

LỜI CẢM ƠN

Lời đầu tiên, nhóm xin được gửi lời cảm ơn chân thành nhất đến PGS. TS Hoàng Văn Dũng. Thầy đã tận tâm hướng dẫn nhóm chúng em từ lúc chọn đề tài cho đến lúc hoàn thành đồ án. Qua những bài học, đóng góp của thầy đã giúp nhóm hiểu hơn về môn học Xử lý ảnh số cũng như trong quá trình hoàn thành đồ án. Cảm ơn những lời khuyên thực tế của thầy đã giúp nhóm đi đúng hướng và hoàn thành đề tài cuối kỳ này.

Do thời gian đợt học lần này đã ngắn đi nên cùng với những kiến thức và bài học tiếp thu được, nhóm đã cố gắng thực hiện đề tài hết mình, cùng nhau thực hiện một cách tốt nhất với khả năng. Chính vì thế trong quá trình đôi khi sẽ có những sai sót không mong muốn, từ đó nhóm hi vọng sẽ nhận được sự góp ý từ thầy để nhóm có thể sửa chữa và rút ra bài học hướng đến mục tiêu là hoàn thiện các đồ án sau này.

Một lần nữa nhóm xin chân thành cảm ơn thầy. Kính chúc thầy thật nhiều sức khỏe, niềm vui trong sự nghiệp trồng người. Nhóm chúng em xin cảm ơn thầy.

TP. Hồ Chí Minh, ngày 22 tháng 12 năm 2025

Nhóm sinh viên thực hiện

Phạm Trần Thiên Đăng

Phạm Võ Nhất Kha

Huỳnh Thanh Nhân

Huỳnh Ngọc Thắng

MỤC LỤC

LỜI CẢM ƠN.....	1
MỤC LỤC	2
DANH MỤC HÌNH ẢNH.....	3
PHẦN MỞ ĐẦU	5
1. Lý do chọn đề tài.....	5
2. Mục tiêu đề tài.....	5
3. Yêu cầu đặt ra.....	5
4. Phạm vi và đối tượng	5
PHẦN NỘI DUNG.....	7
CHƯƠNG 1: CƠ SỞ LÝ THUYẾT	7
1.1. Phần mềm.....	7
1.2. Phần cứng.....	10
CHƯƠNG 2: PHƯƠNG PHÁP THỰC HIỆN	12
2.1. Sơ đồ khối của bài toán.....	12
2.2. Thu thập dữ liệu	12
2.3. Trích xuất dữ liệu, chuẩn hóa và tăng cường.....	19
2.4. Tiền xử lý dữ liệu cho huấn luyện	21
2.5. Model Training (TensorFlow/ Keras).....	24
2.6. Real-time inference	28
CHƯƠNG 3: THỰC NGHIỆM VÀ KẾT QUẢ.....	35
3.1. Thực nghiệm	35
3.2. Kết quả thử nghiệm.....	37
PHẦN KẾT LUẬN	44
1. Đánh giá kết quả.....	44
1.1. Ưu điểm.....	44
1.2. Nhược điểm.....	44
2. Hướng phát triển	44
TÀI LIỆU THAM KHẢO	45
BẢNG KHỐI LƯỢNG CÔNG VIỆC.....	46

DANH MỤC HÌNH ẢNH

Hình 1.1: MediaPipe trích xuất 21 keypoints trên bàn tay và hiển thị dạng Skeleton ...	8
Hình 2.1: Sơ đồ khối bài toán.....	12
Hình 2.2: Chương trình thu thập dữ liệu hình ảnh	13
Hình 2.3: 16 thư mục tương ứng với 16 nhãn của tập dữ liệu.....	14
Hình 2.4: Lưu ảnh bàn tay nhiều người khác nhau trong các điều kiện khác nhau	14
Hình 2.5: Code thực hiện chụp ảnh bàn tay và lưu với kích thước 640x640.....	18
Hình 2.6: Code tiền xử lý trước khi trích xuất tọa độ 21 điểm landmarks.....	20
Hình 2.7: Load data lên google colab.....	22
Hình 2.8: Label encoding	22
Hình 2.9: Train/Validation/ Test split	23
Hình 2.10: Data Augmentation.....	23
Hình 2.11: Model architecture.....	25
Hình 2.12: Train processing	27
Hình 2.13: Evaluation.....	28
Hình 2.14: Save model	28
Hình 2.15: Load model và metadata.....	29
Hình 2.16: GestureVoter class.....	31
Hình 2.17: Hàm normalize features.....	32
Hình 2.18: Hàm map prediction label	33
Hình 2.19: Kiểm tra độ ổn định hướng bàn tay.....	33
Hình 2.20: Dự đoán Gesture.....	33
Hình 2.21: Khởi tạo GestureVoter	33
Hình 2.22: Hậu xử lý dự đoán	34
Hình 2.23: Hiển thị kết quả	34
Hình 3.1: Sơ đồ lắp mạch	37
Hình 3.2: Quá trình xây dựng phần cứng	37
Hình 3.3: Hệ thống đang hoạt động.....	38
Hình 3.4: Ghi log trong quá trình chạy.....	38
Hình 3.5: Khởi động hệ thống bằng cử chỉ tay nhấn “Start”	39
Hình 3.6: Điều khiển quạt hoạt động ở tốc độ 1	39
Hình 3.7: Bật đèn tầng 1	40

Hình 3.8: Bật đèn tầng 2	40
Hình 3.9: Tắt đèn tầng 1	41
Hình 3.10: Xoay quạt	41
Hình 3.11: Đồ thị thể hiện số lỗi nhận diện theo từng nhãn.....	42
Hình 3.12: Ma trận thống kê kết quả dự đoán theo từng nhãn.....	42
Hình 3.13: Đồ thị biểu diễn độ chính xác theo nhãn của hệ thống nhận diện.....	43

PHẦN MỞ ĐẦU

1. Lý do chọn đề tài

Hiện nay với sự phát triển không ngừng nghỉ của khoa học và công nghệ, máy móc hiện tại có thể trở thành một hệ thống thông minh có khả năng tự động hóa, khả năng học hỏi, giải quyết vấn đề và ra quyết định như con người. Trong bối cảnh ấy, các công nghệ về điều khiển không chạm đang dần được ưa chuộng bởi tính tiện lợi và hữu ích của nó trong hầu hết các lĩnh vực. Cùng với sự phát triển của IoT và thị giác máy tính, việc sử dụng ký hiệu tay để điều khiển các thiết bị thông minh trong gia đình, nơi làm việc,... đã và đang là một làn gió mới, giúp chúng ta tương tác với các thiết bị thông minh một cách thuận tiện và linh hoạt hơn.

Trong thực tế, một số môi trường làm việc đặc biệt cần hạn chế tiếp xúc để đảm bảo an toàn, chính vì thế việc tiếp xúc bằng tay trực tiếp với các công tắc vật lý được xem là tối kị. Ngay cả trong đời sống, việc điều khiển thiết bị thông qua nút bấm, remote, thậm chí qua ứng dụng trên điện thoại cũng trở nên bất tiện trong một số trường hợp. Vì thế, việc có một hệ thống cho phép nhìn và nhận dạng các ký hiệu tay để thay đổi công tắc của các thiết bị nhất định là thật sự cần thiết và hữu ích.

2. Mục tiêu đề tài

Nhóm đề xuất mô hình nhận diện ký hiệu tay bằng xử lý ảnh để điều khiển thiết bị IoT như một giải pháp hiện đại, trực quan và hiệu quả. Cụ thể, hệ thống cho phép người dùng sử dụng các cử chỉ tay đơn giản để ra lệnh bật/tắt hoặc điều khiển quạt và đèn trong nhà hay môi trường làm việc. Toàn bộ hệ thống là sự kết hợp của các thiết bị phần cứng (ESP32, camera OV2640, quạt, đèn,...) và mô hình trí tuệ nhân tạo được huấn luyện để nhận diện ký hiệu tay thông qua hình ảnh thu được từ camera OV2640.

3. Yêu cầu đặt ra

- Về độ chính xác: Đảm bảo hệ thống nhận diện chính xác ký hiệu tay với nhiều góc độ và điều kiện môi trường khác nhau để thực hiện đúng yêu cầu của người dùng, hạn chế tối đa tình trạng nhận diện sai gây bất tiện cho người sử dụng.

- Tốc độ xử lý: Hệ thống phải nhận diện được ký hiệu một cách nhanh chóng, không được quá chậm trễ ảnh hưởng đến trải nghiệm người dùng.

4. Phạm vi và đối tượng

Phạm vi:

Mô hình được huấn luyện với 15 ký hiệu tay tương ứng với tập dữ liệu gồm 18.639 ảnh được đánh nhãn (29.648 mẫu sau khi lật ảnh các cử chỉ đối xứng). Hệ thống điều khiển một số chế độ hoạt động đơn giản của các thiết bị như bật/tắt đối với quạt, đèn, thêm điều chỉnh độ mạnh/yếu và hướng đối với quạt, đảm bảo hoạt động ổn định trong môi trường ánh sáng vừa và tốt. Nhằm tránh nhiễu tín hiệu, hệ thống chỉ nhận cử chỉ tay từ một người điều khiển (chỉ lấy cử chỉ tay một người nếu có nhiều người trong khung hình).

Đối tượng:

Đồ án thực hiện dựa trên tập dữ liệu hình ảnh các ký hiệu tay để huấn luyện mô hình cùng với đó là các thuật toán để hỗ trợ xử lý và nhận diện ảnh (tiền xử lý trước khi trích xuất dữ liệu: Adaptive Gamma, lọc Gaussian; tiền xử lý trước khi huấn luyện: Landmark-based Geometric Rotation để xoay hình 2D, Adam Optimizer,...) giúp mô hình hoạt động hiệu quả hơn. Về phần cứng, để phù hợp với quy mô, ESP32-camera, quạt mini (5V-12V), module L298 mạch cầu H điều khiển động cơ DC, PWM và SG90S servo là một trong những sự lựa chọn tối ưu. Về phần mềm, chương trình huấn luyện mô hình được viết bằng ngôn ngữ Python trên Visual Studio Code IDE với các thư viện hỗ trợ mạnh mẽ cho huấn luyện AI như TensorFlow, MediaPipe, Pandas,... Chương trình điều khiển thiết bị phần cứng sẽ triển khai bằng ngôn ngữ lập trình C trên Arduino IDE.

PHẦN NỘI DUNG

CHƯƠNG 1: CƠ SỞ LÝ THUYẾT

1.1. Phần mềm

1.1.1. Xử lý ảnh số

Xử lý ảnh số là quá trình biến đổi một ảnh đầu vào, làm nổi bật các đặc trưng và thông tin cần thiết và hiển thị ra một ảnh mới nhằm thuận tiện cho việc truy xuất thông tin từ ảnh. Một hệ thống xử lý ảnh số gồm các thành phần chính:

- Thu nhận ảnh: Thu nhận ảnh từ thế giới thực và lưu dưới dạng tín hiệu rời rạc qua các thiết bị thu nhận hình ảnh (ngày nay thường là máy ảnh hoặc thiết bị gắn camera).

- Tiền xử lý: Biến đổi bước đầu trên ảnh đầu vào nhằm khử nhiễu, tăng cường chất lượng ảnh, hỗ trợ cho các bước xử lý phức tạp phía sau.

- Trích chọn đặc trưng: Từ ảnh đầu vào, dùng các phương pháp, thuật toán để trích xuất ra tập các đặc trưng có trong ảnh. Mỗi đặc trưng có tính chất khác nhau.

- Phân loại, nhận dạng mẫu: Dùng các phương pháp, kỹ thuật phân tích đặc trưng để phân loại các mẫu thông tin về các nhóm riêng biệt, các phần tử trong một nhóm nhóm có một số tính chất chung.

- Biểu diễn tri thức: Sau khi phân loại và nhận dạng, dữ liệu được biểu diễn dưới dạng tri thức để các hệ thống học máy có thể hiểu ngữ nghĩa của hình ảnh tùy theo sự ứng dụng của từng loại mô hình hay hệ thống trí tuệ nhân tạo.

- Ra quyết định: Khi các mẫu thông tin đã được biểu diễn dưới dạng tri thức, các mô hình sẽ học hỏi và suy luận ra các ngữ nghĩa của hình ảnh tương ứng, từ đó đưa ra các quyết định thực thi các nhiệm vụ tùy theo mục đích phát triển ban đầu.

1.1.2. Ngôn ngữ Python

Ngôn ngữ lập trình Python không còn quá xa lạ với cộng đồng lập trình viên. Python rất mạnh trong lĩnh vực trí tuệ nhân tạo nhờ kho thư viện và framework rộng lớn và người dùng nhiều. Python là ngôn ngữ cấp cao, cú pháp đơn giản, do đó việc học Python vô cùng dễ dàng và một chương trình Python thì ngắn gọn và dễ hiểu.

1.1.3. Thư viện OpenCV

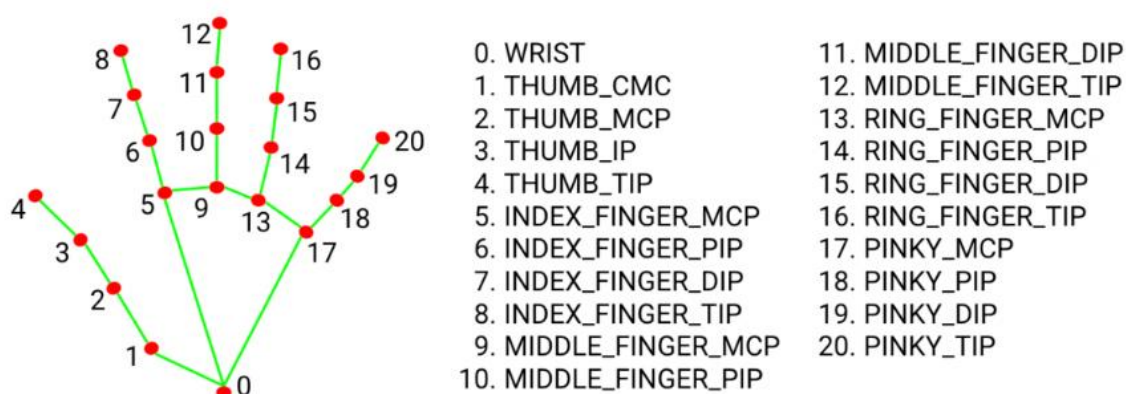
OpenCV là thư viện cần thiết nếu làm việc với các bài toán liên quan đến xử lý ảnh số hay thị giác máy tính,. Phát triển từ năm 1999 bởi Gary Bradsky, OpenCV dần chiếm trở nên phổ biến nhờ mã nguồn mở, hỗ trợ nhiều hệ điều hành khác nhau và hoàn toàn miễn phí. Ngày nay, OpenCV đã phát triển thêm tính năng tăng tốc GPU cho các hoạt động yêu cầu thời gian thực.

1.1.4. Thư viện MediaPipe

MediaPipe là một thư viện học máy mã nguồn mở phát triển bởi Google. Theo Google, thư viện này cực kì nhẹ, chạy ổn định trên hầu hết các loại phần cứng, rất dễ cài đặt và triển khai. MediaPipe cung cấp rất nhiều giải pháp hỗ trợ cho việc phát triển các bài toán học máy nói chung và thị giác máy tính nói riêng, cụ thể như nhận diện khuôn mặt, nhận diện lưới các điểm trên khuôn mặt hay với đồ án này là nhận diện bàn tay. Nhờ có thư viện MediaPipe, vấn đề trích xuất các vị trí landmarks trên bàn tay được xử lý dễ dàng hơn với đầu ra là một mô hình khung xương thể hiện được vị trí 21 landmarks nói với nhau thành một bàn tay hoàn chỉnh.

Tên mẫu thiết bị	Hình dạng đầu vào	Loại lượng tử hoá	Thẻ mô hình	Phiên bản
HandLandmarker (đầy đủ)	192 x 192, 224 x 224	float 16	info	Mới nhất

Gói mô hình điểm mốc của bàn tay phát hiện vị trí điểm chính của 21 tọa độ khớp ngón tay trong các vùng bàn tay đã phát hiện. Mô hình này được huấn luyện trên khoảng 30.000 hình ảnh thực tế, cũng như một số mô hình tay tổng hợp được kết xuất trên nhiều nền.



Hình 1.1: MediaPipe trích xuất 21 keypoints trên bàn tay và hiển thị dạng Skeleton

1.1.5. TensorFlow Framework

TensorFlow là thư viện mã nguồn mở được cộng đồng AI và Machine Learning rất ưa chuộng. Trong quá trình nghiên cứu lĩnh vực trí tuệ nhân tạo, Google nhận thấy

Deep Learning có khả năng xử lý dữ liệu lớn hiệu quả hơn các thuật toán học máy khác. Năm 2015 phiên bản TensorFlow đầu tiên ra đời, sau đó tiếp tục cải tiến và xuất bản phiên bản ổn định vào năm 2017.

Với đồ án, Tensorflow (2.16.1) được xem là sự lựa chọn hợp lý nhất hiện tại nhờ tính tương thích tốt với MediaPipe (ổn định hơn PyTorch) và mạnh với bài toán phân loại. Ngoài ra, TensorFlow còn hỗ trợ nhiều định dạng model, dễ sử dụng và triển khai.

1.1.6. Thư viện Keras

Keras cũng là thư viện mã nguồn mở được Francois Chollet phát triển năm 2015, tương thích cực tốt với TensorFlow và cũng dễ tiếp cận kể cả với người mới. Keras cung cấp API đơn giản để làm việc với mạng nơ-ron nhân tạo, rút gọn code thay vì phải cấu hình nhiều tham số như trước kia, đơn giản hóa việc tương tác với model đồng thời cũng hỗ trợ nhiều định dạng model. Năm 2017, Google quyết định tích hợp thư viện Keras vào TensorFlow (từ bản 2.x). Hiện tại với TensorFlow 2.16.1, chúng ta chỉ cần “import” như sau: *import tensorflow as tf* là có thể sử dụng keras bằng cách gọi *tf.keras*.

1.1.7. Thư viện Pandas

Pandas là thư viện ra đời vào năm 2008 bởi Wes McKinney. Pandas tạo được DataFrame từ dữ liệu, hỗ trợ tạo file csv. Liên hệ trong đồ án, model cần học dữ liệu gồm feature là 46 số (tọa độ keypoint), nhãn tên cử chỉ, “has_hand” dạng boolean và handedness (trái hoặc phải). Pandas sẽ lưu thông tin này vào một DataFrame và lưu dạng csv. Nhờ đó, chúng ta dễ dàng thống kê được số liệu và cũng giúp việc huấn luyện model trở nên dễ dàng hơn.

1.1.8. Visual Studio Code IDE

Hiện nay, ngôn ngữ lập trình Python đang dần trở nên phổ biến kéo theo sự phát triển của các IDE nhằm hỗ trợ tốt nhất cho cộng đồng lập trình viên Python. Visual Studio Code cũng là một trong những môi trường phát triển tích hợp mạnh mẽ, hỗ trợ tốt cho Python. Được phát triển theo hướng mã nguồn mở, Microsoft đã đưa Visual Studio Code vào danh sách các IDE được sử dụng phổ biến nhất hiện tại nhờ tính linh hoạt từ những tiện ích mở rộng.

1.1.9. Ngôn ngữ lập trình C và Arduino IDE

Ngôn ngữ lập trình C là ngôn ngữ lập trình cấp thấp nhưng hiệu năng cao, tốc độ nhanh (đáp ứng gần thời gian thực). Chính vì thế, các chương trình lập trình nhúng thường sử dụng ngôn ngữ C thay vì các ngôn ngữ cấp cao như Python. Ngay các vi điều khiển ESP32 cũng được thiết kế để có thể lập trình bằng ngôn ngữ C.

Đi cùng với đó là Arduino IDE, môi trường phát triển dành cho lập trình nhúng, hỗ trợ cho nhiều vi điều khiển, trong đó có ESP32. Arduino IDE cũng là một IDE dễ sử dụng, không mất nhiều thời gian cấu hình, giao diện thì đơn giản và nhiều thư viện.

1.2. Phần cứng

1.2.1. ESP32 / ESP32-CAM

ESP32 là một vi điều khiển mạnh mẽ với khả năng tương thích cao. Một ESP32 cơ bản có khoảng 34 chân GPIO đa dụng cùng nhiều cổng giao tiếp tích hợp như 3 cổng UART, 3 đến 4 cổng SPI, 2 cổng I2C, 2 cổng DAC, 18 kênh ADC 12-bit, 2 cổng I2S, Ethernet MAC. ESP32 cực kì dễ sử dụng và tiếp cận. ESP32 có tích hợp Wi-Fi và Bluetooth, đơn giản hóa việc kết nối với các thiết bị khác.

Đối với đồ án của nhóm, ESP32 là lựa chọn tối ưu để xử lý ảnh mức cơ bản, đảm nhận xử lý tín hiệu điều khiển quạt và đèn, có hỗ trợ tích hợp Wi-Fi thuận tiện cho việc kết nối và tương thích tốt với thiết bị thu hình của hệ thống là camera OV2640. Ngoài ra, ESP32 có giá khá phải chăng, phù hợp với sinh viên. Một số ESP32 còn tích hợp thêm camera (thường là OV2640) gọi là ESP32-CAM.

1.2.2. Camera OV2640

OV2640 là module camera có độ phân giải tối đa 2MP, thường dùng với ESP32-CAM. Với bài toán nhận diện cử chỉ tay, độ phân giải của OV2640 là vừa đủ để xử lý. Lý do quan trọng nhất để lựa chọn OV2640 là camera này ngày nay được tích hợp chung với các ESP32, có cả thư viện hỗ trợ, do đó việc sử dụng OV2640 là tối ưu.

1.2.3. Servo SG90S

Servo là thiết bị thực hiện chuyển động quay của quạt trong hệ thống. Kết hợp SG90S với ESP32 giúp cho việc lập trình dễ dàng hơn và phản hồi cũng nhanh hơn do SG90S nhận tín hiệu điều khiển PWM từ ESP32 một cách chính xác và ổn định, do có kích thước thì nhỏ, không cần nguồn có công suất lớn nên rất an toàn.

1.2.4. Mạch điều khiển động cơ DC L298N

L298N là một module điều khiển động cơ dạng cầu H kép sử dụng chip IC L298, cho phép điều khiển độc lập hai động cơ DC hoặc một động cơ bước. Nhờ khả năng đảo chiều dòng điện, mạch điều khiển L298N được sử dụng để điều khiển tốc độ và sự quay của quạt đồng thời điều khiển bật/tắt đèn. L298N còn có tản nhiệt, diode bảo vệ chống dòng ngược từ động cơ và bộ điều chỉnh điện áp.

1.2.5. AMS1117

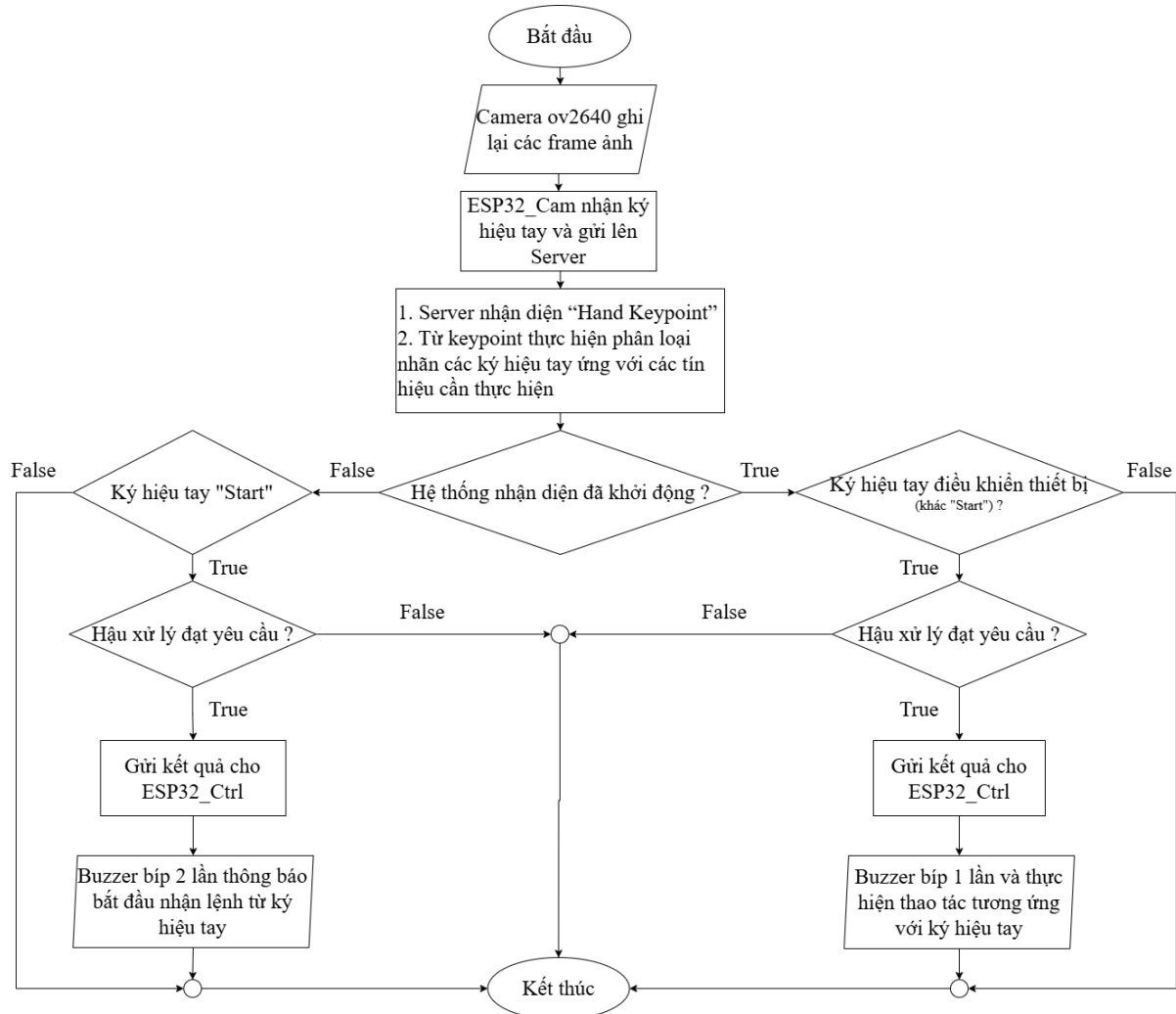
AMS1117 là một loại IC ổn áp tuyến tính 3 chân rất phổ biến có vai trò chuyển đổi điện áp DC từ mức cao xuống mức thấp hơn và ổn định hơn. Do hệ thống gồm nhiều thiết bị với các mức điện áp khác nhau (ESP32 dùng 3.3V, đèn dùng 5V và quạt dùng 12V), vì thế cần một thiết bị ổn áp. AMS1117 tương thích với ESP32.

1.2.6. Quạt, đèn và buzzer

Là những thiết bị chính, thực hiện các thao tác được điều khiển bằng ký hiệu tay. Nhóm quyết định chọn quạt (5V – 12V), đèn (5V) và buzzer để phát âm thanh, giúp người sử dụng biết ký hiệu tay có được nhận diện hay chưa để điều chỉnh cử chỉ.

CHƯƠNG 2: PHƯƠNG PHÁP THỰC HIỆN

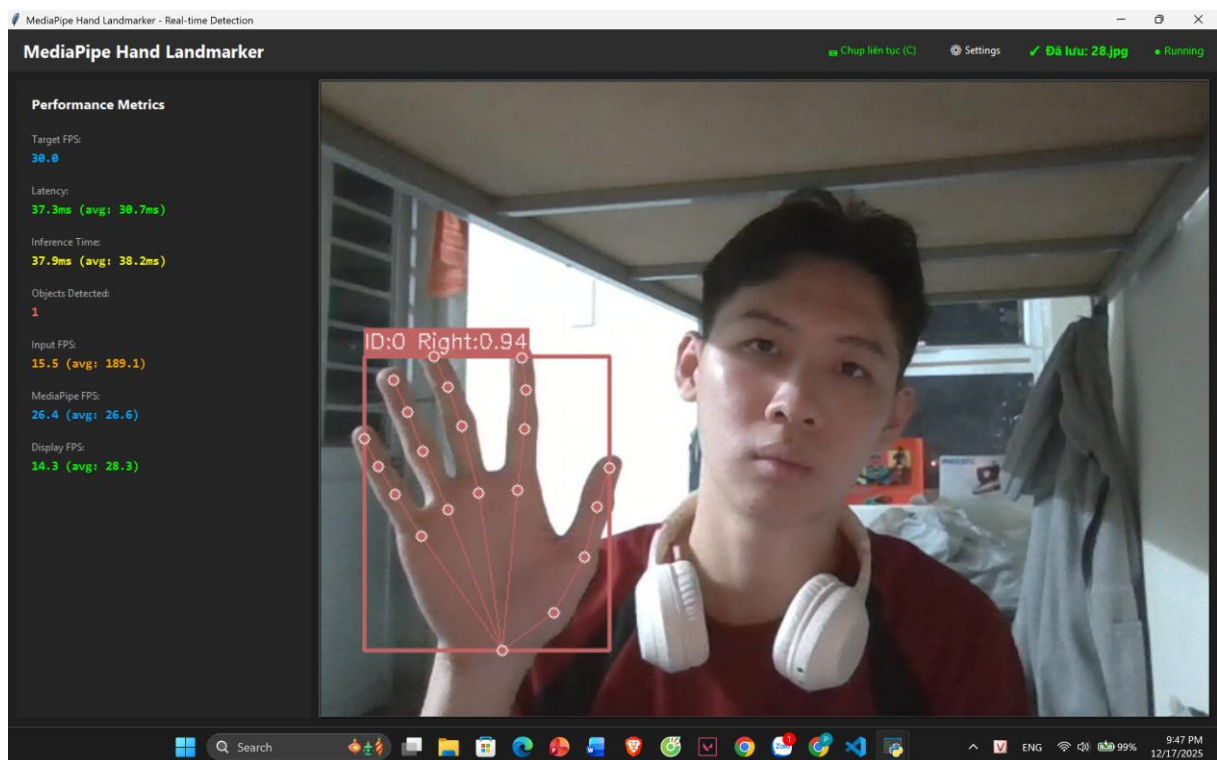
2.1. Sơ đồ khối của bài toán



Hình 2.1: Sơ đồ khối bài toán

2.2. Thu thập dữ liệu

Một hệ thống xử lý ảnh số thường bắt đầu với việc thu nhập dữ liệu, cụ thể là hình ảnh. Để thuận tiện cho việc lấy hình ảnh và đảm bảo thu được bộ dữ liệu tương đối ổn định cho bước huấn luyện, nhóm đã xây dựng một ứng dụng nhỏ có sử dụng thư viện của MediaPipe để trích xuất bộ khung xương bàn tay chứa 21 điểm quan trọng và hiển thị real-time, đồng thời chụp lại ảnh bàn tay mỗi khi hệ thống trích xuất thành công và lưu với kích thước 640x640. Chương trình chỉ chụp ảnh bàn tay khi hệ thống đã trích xuất được tọa độ 21 điểm giúp hạn chế vấn đề không thể nhận diện đủ 21 điểm từ ảnh ở các bước kế tiếp.



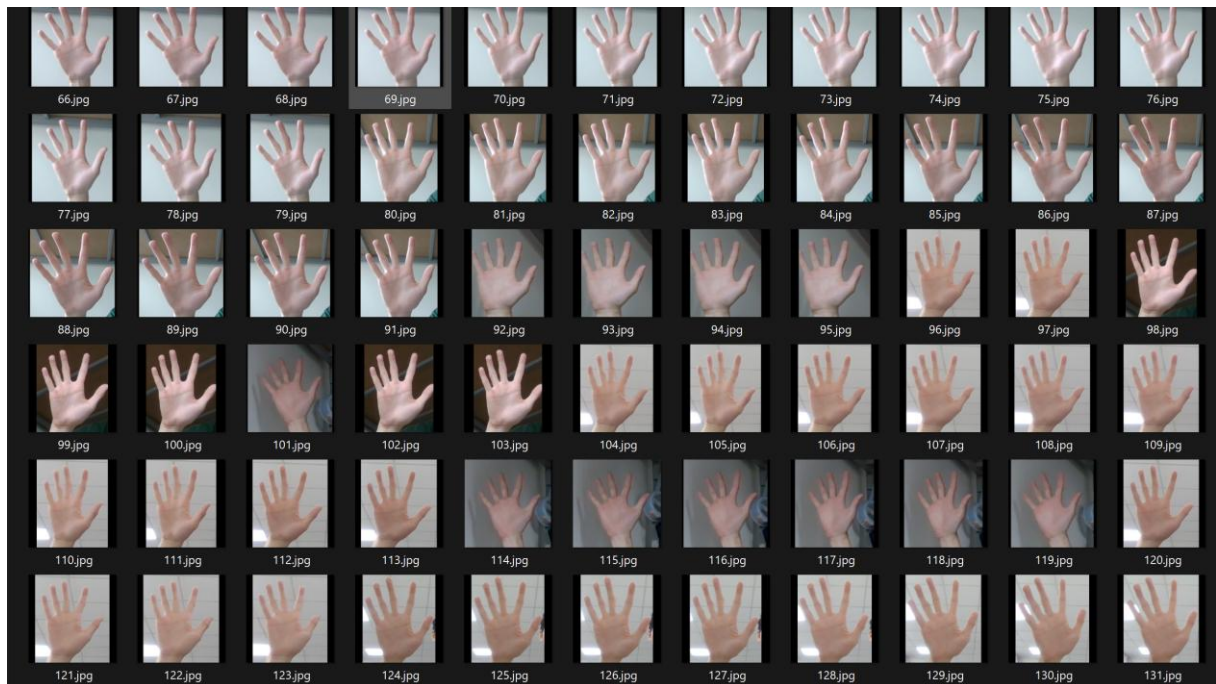
Hình 2.2: Chương trình thu thập dữ liệu hình ảnh

Khi chụp ảnh, mỗi loại cử chỉ tay sẽ được lưu vào một thư mục với nhãn khác nhau, tổng cộng có 16 thư mục tương ứng với 16 nhãn, trung bình mỗi thư mục lưu khoảng 1000 đến 1200 ảnh của loại cử chỉ theo nhãn đó. Nhằm tăng tính đa dạng, mỗi loại cử chỉ đều sẽ lưu ảnh bàn tay của các thành viên khác nhau trong nhóm và trong nhiều điều kiện môi trường, ánh sáng, góc độ khác nhau.

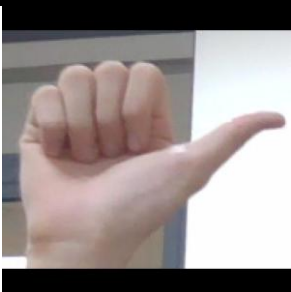
Các cử chỉ tay được chia thành 2 loại: đối xứng và bất đối xứng. Các cử chỉ tay đối xứng như nhãn “S_Start”, nhãn “S_Light1On” sẽ thực hiện thu trên tay phải, các cử chỉ tay bất đối xứng (cần xác định hướng trái/phải) như sẽ được thu trên cả 2 tay như “A_LH_FanRight” hay “A_RH_FanLeft”. Về ý nghĩa tiền tố trong tên của các nhãn, S viết tắt của “symmetric” (đối xứng), A viết tắt của “asymmetric” (bất đối xứng), LH và RH lần lượt là “left hand” và “right hand” (tay trái, tay phải).

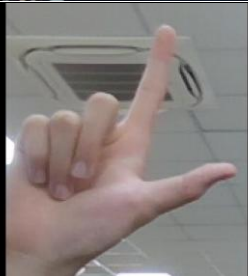
Name	Date modified	Type	Size
A_LH_FanLeft	12/10/2025 9:18 PM	File folder	
A_LH_FanRight	12/10/2025 9:18 PM	File folder	
A_RH_FanLeft	12/10/2025 12:50 PM	File folder	
A_RH_FanRight	12/10/2025 1:17 PM	File folder	
S_FanDown	12/10/2025 3:43 PM	File folder	
S_FanOff	12/9/2025 1:29 PM	File folder	
S_FanSpeed1	12/10/2025 3:46 PM	File folder	
S_FanSpeed2	12/10/2025 8:29 PM	File folder	
S_FanSpeed3	12/10/2025 8:29 PM	File folder	
S_FanUp	12/10/2025 3:06 PM	File folder	
S_Light1Off	12/10/2025 8:31 PM	File folder	
S_Light1On	12/10/2025 8:31 PM	File folder	
S_Light2Off	12/10/2025 8:31 PM	File folder	
S_Light2On	12/10/2025 8:31 PM	File folder	
S_Nothing	12/11/2025 1:23 PM	File folder	
S_Start	12/10/2025 12:17 AM	File folder	

Hình 2.3: 16 thư mục tương ứng với 16 nhãn của tập dữ liệu



Hình 2.4: Lưu ảnh bàn tay nhiều người khác nhau trong các điều kiện khác nhau

STT	Nhãn	Hành động thực hiện	Hình ảnh đại diện	Số lượng ảnh
1	A_LH_FanLeft	Quay quạt sang trái bằng tay trái		1116
2	A_LH_FanRight	Quay quạt sang phải bằng tay trái		1144
3	A_RH_FanLeft	Quay quạt sang trái bằng tay phải		1102
4	A_RH_FanRight	Quay quạt sang phải bằng tay phải		1401
5	S_FanDown	Điều khiển quạt hướng xuống		1411
6	S_FanOff	Tắt quạt		1028

7	S_FanSpeed1	Bật quạt mức 1		1055
8	S_FanSpeed2	Bật quạt mức 2		1057
9	S_FanSpeed3	Bật quạt mức 3		1011
10	S_FanUp	Điều chỉnh quạt hướng lên		1501
11	S_Light1Off	Tắt đèn tầng 1		1058
12	S_Light1On	Bật đèn tầng 1		1003

13	S_Light2Off	Tắt đèn tầng 2		1076
14	S_Light2On	Bật đèn tầng 2		1057
15	S_Nothing	Không có cử chỉ tay		1618
16	S_Start	Khởi động nhận diện		1001

Bảng 2.1: Thống kê dữ liệu thu thập

```

1407 def save_hand_image(frame, min_x, min_y, max_x, max_y):
1408     """
1409     Cắt và lưu ảnh bàn tay với kích thước 640x640
1410
1411     Args:
1412         frame: Frame gốc (BGR)
1413         min_x, min_y, max_x, max_y: Tọa độ bounding box của bàn tay
1414     """
1415     global save_image_counter, last_save_time, notification_label, notification_timer, root
1416     global missing_image_numbers
1417
1418     # Kiểm tra khoảng thời gian giữa các lần lưu
1419     current_time = time.time()
1420     if current_time - last_save_time < SAVE_INTERVAL:
1421         return
1422
1423     try:
1424         # Tạo thư mục nếu chưa có
1425         os.makedirs(SAVE_DIR, exist_ok=True)
1426
1427         # Cắt ảnh từ bounding box
1428         padding = 20
1429         frame_h, frame_w = frame.shape[:2]
1430
1431         # Tính toán tọa độ với padding
1432         crop_x1 = max(0, min_x - padding)
1433         crop_y1 = max(0, min_y - padding)
1434         crop_x2 = min(frame_w, max_x + padding)
1435         crop_y2 = min(frame_h, max_y + padding)
1436
1437         # Cắt ảnh
1438         hand_crop = frame[crop_y1:crop_y2, crop_x1:crop_x2]
1439
1440         if hand_crop.size == 0:
1441             return
1442
1443         # Resize về 640x640
1444         target_size = 640
1445         h, w = hand_crop.shape[:2]
1446
1447         # Tính scale để fit vào 640x640
1448         scale = min(target_size / w, target_size / h)
1449         new_w = int(w * scale)
1450         new_h = int(h * scale)
1451
1452         # Resize
1453         hand_resized = cv2.resize(hand_crop, (new_w, new_h), interpolation=cv2.INTER_AREA)
1454
1455         # Tạo ảnh 640x640 với background đen
1456         hand_final = np.zeros((target_size, target_size, 3), dtype=np.uint8)
1457
1458         # Đặt ảnh đã resize vào giữa
1459         y_offset = (target_size - new_h) // 2
1460         x_offset = (target_size - new_w) // 2
1461         hand_final[y_offset:y_offset+new_h, x_offset:x_offset+new_w] = hand_resized

```

```

1463     # Tìm số counter để lưu: ưu tiên dùng số còn thiếu trước
1464     counter_to_use = None
1465
1466     # Ưu tiên 1: Dùng số từ danh sách các số còn thiếu
1467     if missing_image_numbers:
1468         counter_to_use = missing_image_numbers.pop(0) # Lấy số đầu tiên và xóa khỏi list
1469     else:
1470         # Ưu tiên 2: Dùng số tiếp theo từ counter
1471         # Kiểm tra xem số này đã tồn tại chưa (phòng trường hợp có file mới được thêm vào từ bên ngoài)
1472         temp_counter = save_image_counter
1473         max_attempts = 1000
1474         attempts = 0
1475
1476         while attempts < max_attempts:
1477             filename_check = f"{temp_counter}.jpg"
1478             filepath_check = os.path.join(SAVE_DIR, filename_check)
1479
1480             # Nếu file chưa tồn tại, dùng số này
1481             if not os.path.exists(filepath_check):
1482                 counter_to_use = temp_counter
1483                 save_image_counter = temp_counter + 1 # Cập nhật counter cho lần sau
1484                 break
1485
1486             # Nếu file đã tồn tại, thử số tiếp theo
1487             temp_counter += 1
1488             attempts += 1
1489
1490         if attempts >= max_attempts:
1491             print(f"X Không thể tìm được tên file trống sau {max_attempts} lần thử")
1492             return
1493
1494     # Tạo tên file và đường dẫn
1495     filename = f"{counter_to_use}.jpg"
1496     filepath = os.path.join(SAVE_DIR, filename)
1497
1498     # Kiểm tra lại một lần nữa để đảm bảo an toàn (phòng trường hợp có race condition)
1499     if os.path.exists(filepath):
1500         print(f"⚠ File {filename} đã tồn tại, bỏ qua lần lưu này")
1501         return
1502
1503     # Lưu ảnh
1504     cv2.imwrite(filepath, hand_final)
1505
1506     # Cập nhật thời gian
1507     last_save_time = current_time
1508
1509     # Hiển thị thông báo
1510     if notification_label:
1511         notification_label.config(text=f"✓ Đã lưu: {filename}", fg="#00ff00")
1512
1513     # Hủy timer cũ nếu có
1514     if notification_timer:
1515         root.after_cancel(notification_timer)
1516
1517     # Tự động ẩn thông báo sau 2 giây và quay lại hiển thị trạng thái auto-save
1518     def restore_auto_save_status():
1519         if notification_label:
1520             if SAVE_HAND_IMAGES:
1521                 notification_label.config(text="✓ Auto-save: ON", fg="#00ff00")
1522             else:
1523                 notification_label.config(text="X Auto-save: OFF", fg="#ff6b6b")
1524         notification_timer = root.after(2000, restore_auto_save_status)
1525
1526     print(f"✓ Đã lưu ảnh: {filepath}")
1527
1528 except Exception as e:
1529     print(f"X Lỗi khi lưu ảnh: {e}")

```

Hình 2.5: Code thực hiện chụp ảnh bàn tay và lưu với kích thước 640x640

2.3. Trích xuất dữ liệu, chuẩn hóa và tăng cường

Sau khi đã thu thập đầy đủ dữ liệu ảnh cho từng nhãn, tiếp theo cần trích xuất tọa độ của từng ảnh trong tập dữ liệu và lưu dưới dạng file .csv. Vì ảnh thu thập có kích thước 640x640, số điểm ảnh cần xử lý rất lớn nên cần chuẩn hóa, loại bỏ các nhiễu từ môi trường để chỉ xử lý các đặc trưng cần thiết, cụ thể là 21 điểm, mỗi điểm biểu diễn tọa độ theo x và y, thêm 4 điểm đại diện trục tọa độ, tổng là 46 số.

Chương trình trích xuất tọa độ, chuẩn hóa và tăng cường thực hiện theo luồng sau:

- Phân loại gesture (cử chỉ tay) đối xứng và bất đối xứng dựa trên nhãn và đếm số ảnh trong mỗi thư mục.
- Tiền xử lý ảnh:
 - Đảm bảo đồng nhất kích thước ảnh về 640x640.
 - Chia cho 255.0 để đưa toàn bộ điểm ảnh về khoảng [0,1].
 - Cân bằng trắng để khử lỗi ám màu, đưa màu da tay về chuẩn thực tế, đồng thời bù sáng nhẹ toàn cục khoảng 10% để hỗ trợ MediaPipe thấy rõ các nếp gấp khớp tay.
 - Dùng Adaptive Gamma để triệt tiêu sự khác biệt về độ tương phản giữa ảnh sáng và ảnh tối nhằm đảm bảo hình dạng bộ khung xương luôn nhất quán bất kể môi trường chụp.
 - Cuối cùng, dùng lọc Gaussian (hàm *cv2.GaussianBlur(img, (3, 3), 0)*) để làm trơn, khử nhiễu ảnh kỹ thuật số cơ bản.

```

73 def tien_xu_ly(img):
74     if img is None:
75         return None
76
77     img = cv2.resize(img, (640, 640))
78     img = img.astype(np.float32) / 255.0
79
80     # White balance (Cân bằng trắng)
81     mean = img.mean(axis=(0, 1), keepdims=True)
82     gray = mean.mean()
83     scale = np.clip(gray / (mean + 1e-6), 0.9, 1.1)
84     img = np.clip(img * scale, 0, 1)
85
86     # Bù sáng nhẹ toàn cục (10%)
87     img = np.clip(img * 1.1, 0, 1)
88
89     # Gamma thích nghi (Adaptive Gamma)
90     avg = img.mean()
91     gamma = np.clip(1.4 - avg, 0.9, 1.4)
92     img = np.power(img, 1/gamma)
93
94     # Denoise bằng Gaussian Blur 3x3
95     img = cv2.GaussianBlur(img, (3, 3), 0)
96
97     return (img * 255).astype(np.uint8)

```

Hình 2.6: Code tiền xử lý trước khi trích xuất tọa độ 21 điểm landmarks

- Nhận diện 21 điểm quan trọng (gọi là 21 landmarks) bằng MediaPipe và xử lý nhãn S_Nothing bằng cách gán “vector zero” và đặt “has_hand=0”.
- Xác thực dữ liệu: Chương trình thực hiện kiểm tra ảnh thu được có chứa chỉ đúng một tay hay không, kiểm tra handedness có khớp với nhãn không, có trích xuất đủ 21 điểm (landmarks) không. Một số trường hợp nhận diện sai hoặc MediaPipe không trích xuất đủ tọa độ 21 điểm (landmarks), chương trình sẽ bỏ qua ảnh đó, tức là không trích xuất tọa độ cho ảnh đó.
- Tăng cường dữ liệu ảnh đối xứng: Nếu là ảnh cử chỉ đối xứng, chương trình thực hiện thêm bước đảo ngược từng ảnh trong nhãn, làm nhân đôi dữ liệu của nhãn đó. Các ảnh được đảo ngược tương đương với cử chỉ của nhãn đó

nhưng thực hiện bằng tay trái, mục đích là đảm bảo hệ thống có thể nhận diện được các nhãn khi ra lệnh bằng cả hai tay.

- Dịch tọa độ để đồng nhất tọa độ trong từng ảnh bằng cách lấy điểm cổ tay trong từng ảnh làm gốc. Sau đó, chia cho giá trị tọa độ lớn nhất trong 21 điểm để chuẩn hóa giá trị tọa độ về khoảng $[-1.0, 1.0]$, mục đích là xóa bỏ sự khác biệt về kích thước bàn tay trong các ảnh khác nhau, đảm bảo mô hình học không bị nhiễu bởi khoảng cách camera. Bổ sung thêm hai trục để xác định hướng của cử chỉ.
- Chuẩn hóa dữ liệu cuối cùng được làm phẳng thành mảng một chiều gồm 42 số đầu đại diện cho 21 điểm với cả tọa độ theo trục x và trục y, 4 số cuối biểu diễn trục x và trục y.
- Lưu dưới dạng file .csv với tên đầy đủ là “dataset.csv”. Việc lưu ra file csv giúp cho quá trình huấn luyện được nhanh chóng hơn, tránh việc chạy lại MediaPipe mỗi vòng lặp huấn luyện, từ đó giảm chi phí và tài nguyên.
- Cuối cùng, để thuận tiện cho việc theo dõi kết quả trích xuất, chương trình sẽ thống kê lại tổng số ảnh, ảnh đã xử lý, ảnh bị bỏ qua, số mẫu thu được, số loại cử chỉ. Chương trình sẽ in ra các ảnh bị bỏ qua theo từng loại cử chỉ trong lúc chạy và thống kê đầy đủ lần nữa sau khi xuất file csv.

2.4. Tiền xử lý dữ liệu cho huấn luyện

Mục đích của xử lý tiền dữ liệu là chuẩn hóa dữ liệu, lọc và lựa chọn những bức ảnh tốt nhất nhằm mục đích chuẩn bị data cho quá trình training model. Quy trình thực hiện như sau:

Bước 1: Load data

Đưa dữ liệu từ file dataset.csv lên Google Colab để chuẩn bị dữ liệu cho quá trình huấn luyện. Sau đó sẽ tiến hành kiểm tra số lượng đặc trưng (features). Tiếp theo là làm sạch dữ liệu và tách dữ liệu thành X (đặc trưng) và Y (nhãn dán) để chuẩn bị tốt cho việc huấn luyện mô hình.

Thực hiện:

```
df = pd.read_csv(CSV_PATH)

feat_cols = [c for c in df.columns if c.startswith("feat_")]
if len(feat_cols) != 46:
    raise ValueError(f"Dataset có {len(feat_cols)} features, cần đúng 46")

df = df.dropna(subset=feat_cols + ["label"]).reset_index(drop=True)

X = df[feat_cols].astype(np.float32).values
y_str = df["label"].astype(str).values
```

Hình 2.7: Load data lên google colab

Bước 2: Label encoding

Vì máy học không hiểu chữ, chỉ học được số nên đây là một bước vô cùng quan trọng để chuyển nhãn dạng chữ (string) sang nhãn dạng số (integer) để mô hình Machine Learning có thể học được.

Thực hiện:

```
# 2) LABEL ENCODER
# =====
le = LabelEncoder()
y = le.fit_transform(y_str)
num_classes = len(le.classes_)
```

Hình 2.8: Label encoding

Bước 3: Train/Validation/ Test split

Bước này dùng để chia tập dữ liệu thành thành ba tập: train/validation/test nhằm huấn luyện, điều chỉnh và đánh giá mô hình. Ở lần thứ nhất chúng ta chia làm hai tập: Train và Temp: 70% Train dùng để huấn luyện mô hình và 30% Temp dùng để chia tiếp thành Val và Test. Chia lần hai thành Validation và Test với 15% cho mỗi tập. Sở dĩ chúng ta không chia luôn trong một lần mà tách ra làm hai là bởi Validation tham gia trong quá trình điều chỉnh và Test phải hoàn toàn độc lập. Chính vì thế khi chia ra 2 lần sẽ giúp chúng ta kiểm soát tốt hơn.

Thực hiện:

```

# 3) SPLIT
# =====
X_train, X_temp, y_train, y_temp = train_test_split(
    X, y, test_size=0.30, random_state=SEED, stratify=y
)

X_val, X_test, y_val, y_test = train_test_split(
    X_temp, y_temp, test_size=0.50, random_state=SEED, stratify=y_temp
)

print("Train / Val / Test:", X_train.shape, X_val.shape, X_test.shape)

```

Hình 2.9: Train/Validation/ Test split

Bước 4: Data augmentation

Dùng để tăng cường dữ liệu cho tập train nhằm giúp mô hình học tốt hơn, tổng quát tốt hơn và giảm overfitting, đặc biệt phù hợp với bài toán bài toán cử chỉ tay. Ở bước này sẽ tạo thêm một tập dữ liệu huấn luyện giả lập từ dữ liệu thật, giúp mô phỏng các biến đổi tự nhiên có thể xảy ra trong quá trình áp dụng vào thực tế như xoay tay, nhiễu do landmark hay sai số camera. Điều này giúp mô hình không phụ thuộc vào 1 tư thế tay nhất định, góp phần cải thiện chất lượng đồ án.

Thực hiện:

```

# 4) DATA AUGMENTATION (TRAIN ONLY)
# =====
def rotate_landmarks(batch, max_deg):
    angle = np.deg2rad(np.random.uniform(-max_deg, max_deg))
    cos, sin = np.cos(angle), np.sin(angle)

    rot = np.array([[cos, -sin],
                    [sin, cos]], dtype=np.float32)

    batch_aug = batch.copy()

    # landmarks: feat_0 .. feat_41 → (21 points)
    for i in range(0, 42, 2):
        pts = batch[:, i:i+2]
        batch_aug[:, i:i+2] = pts @ rot.T

    return batch_aug

def noise_landmarks(batch, std):
    noise = np.random.normal(0, std, batch.shape).astype(np.float32)
    return batch + noise

def augment_train_data(X, y, times=1):
    X_aug, y_aug = [X], [y]

    for _ in range(times):
        X_rot = rotate_landmarks(X, ROTATION_DEG)
        X_noisy = noise_landmarks(X_rot, NOISE_STD)

        X_aug.append(X_noisy)
        y_aug.append(y)

    return np.vstack(X_aug), np.hstack(y_aug)

X_train_aug, y_train_aug = augment_train_data(
    X_train, y_train, times=AUGMENT_TIMES
)

print("🔥 After augmentation:", X_train_aug.shape)

```

Hình 2.10: Data Augmentation

2.5. Model Training (TensorFlow/ Keras)

Model Training là bước quan trọng và cốt lõi nhất trong quá trình xây dựng một mô hình học máy. Model Training nhằm tối ưu các trọng số của mạng nơ-ron để mô hình có thể học được những đặc trưng của đầu vào và nhãn, từ đó dự đoán chính xác được cử chỉ động tác trên dữ liệu mới được đưa vào.

Quy trình thực hiện:

Bước 1: Model architecture

Về bản chất kiến trúc này là một mạng nơ-ron truyền thẳng (Feedforward Neural Network) dùng để phân biệt cử chỉ từ 46 đặc trưng từ tập dữ liệu thu được. Đầu tiên chúng ta cố định RANDOM SEED qua lệnh `tf.keras.utils.set_random_seed(SEED)` nhằm đảm bảo kết quả train có thể lặp lại.

`Layers.Input(shape=(46,))` dùng để khởi tạo mẫu đầu vào là vector 46 chiều bao gồm: tọa độ (x,y) của 21 điểm landmark tay và 2 vector hướng bàn tay, tức $21 \times 2 + 2 \times 2 = 46$ đặc trưng.

Đầu tiên chúng ta thiết kế 1 lớp layer (`layers.Dense(256)`) để học quan hệ phức tạp giữa các điểm landmark. Dùng hàm `layers.BatchNormalization` và `layers.Dropout` để chuẩn hóa đầu ra và chống overfitting. Lớp đầu tiên dùng để học những đặc trưng cấp thấp đến trung bình.

Ở lớp thứ hai (`layers.Dense(128)`) dùng để học những đặc trưng cụ thể hơn, trừu tượng hơn so với lớp thứ nhất. Dropout khi này sẽ giảm nhẹ hơn vì thông tin đã được cô đọng ở lớp thứ nhất. Lớp thứ hai sẽ học những đặc trưng cấp cao hơn.

Ở lớp thứ ba (`layers.Dense(64)`) dùng để tổng hợp những đặc trưng cuối và chuẩn bị cho phân loại.

Ở lớp cuối cùng (`layers.Dense(num_classes, activation="softmax")`) dùng để trả về xác suất từng lớp với xác suất từng lớp bằng 1.

```

# 6) MODEL
# =====
tf.keras.utils.set_random_seed(SEED)

model = models.Sequential([
    layers.Input(shape=(46,)),

    layers.Dense(256),
    layers.BatchNormalization(),
    layers.Activation("relu"),
    layers.Dropout(0.4),

    layers.Dense(128),
    layers.BatchNormalization(),
    layers.Activation("relu"),
    layers.Dropout(0.3),

    layers.Dense(64, activation="relu"),
    layers.Dense(num_classes, activation="softmax")
])

```

Hình 2.11: Model architecture

Bước 2: Hyperparameters

Hyperparameters huấn luyện

BATCH_SIZE = 128

EPOCHS = 120

LR = 1e -3

Hyperparameters chia dữ liệu

SEED = 42

Hyperparameters Data Augmentation

ROTATION_DEG = 8

NOISE_STD = 0.003

AUGMENT_TIMES = 1

Hyperparameters kiến trúc mô hình

Layers.Dense(256)

Layers.Dense(128)

Layers.Dense(64)

Hyperparameters Regularization

Layers.Dropout(0.4)

Layers.Dropout(0.3)

Bước 3: Training

Xây dựng `model.compile()` để cấu hình huấn luyện. Sử dụng thuật toán tối ưu gradient kết hợp Momentum và Adaptive phù hợp với dữ liệu nhiễu và bài toán phân loại nhiều lớp.

Dữ liệu trong dataset có thể không cân bằng, với những class ít mẫu sẽ dễ khiến hệ thống học kém, dẫn đến features đó nhận dạng kém. Chính vì thế cần phải xử lý mất cân bằng dữ liệu thông qua tính CLASS WEIGHT.

Một trong những phần không thể thiếu của quá trình training là quá trình CALLBACKS điều khiển quá trình train. Hàm `ModelCheckpoint` giúp lưu model tốt nhất với các chức năng như theo dõi `val_accuracy`, chỉ lưu model tốt nhất và tránh overfitting. Hàm `EarlyStopping` có chức năng dừng train để tiết kiệm thời gian khi 15 epoch liên tiếp không cải thiện và sẽ quay lại trọng số tốt nhất. Hàm `ReduceLROnPlateau` giúp giảm learning rate.

Để huấn luyện mô hình, hàm `model.fit()` được sử dụng với các thành phần: `X_train_aug`, `y_train_aug` để giúp model tổng quát hóa tốt hơn và giảm overfitting; `validation_data` dùng để theo dõi `val_loss` và `val_accuracy`, cùng với đó là dùng làm điều kiện cho callback; `epochs` là số vòng lặp huấn luyện tối đa nhưng thực tế có thể dừng sớm hơn nhờ `EarlyStopping`; `batch_size` là số mẫu mỗi lần cập nhật gradient, nó ảnh hưởng đến tốc độ và độ ổn định của mô hình; `class_weight` được dùng để cân bằng dữ liệu, class hiếm sẽ được ưu tiên học hơn so với những class dễ nhận diện hơn.

```

model.compile(
    optimizer=tf.keras.optimizers.Adam(LR),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

classes = np.unique(y_train_aug)
weights = compute_class_weight(
    class_weight="balanced",
    classes=classes,
    y=y_train_aug
)
class_weight = {int(c): float(w) for c, w in zip(classes, weights)}

# =====
# 7) CALLBACKS
# =====
checkpoint_cb = callbacks.ModelCheckpoint(
    BEST_KERAS_PATH,
    monitor="val_accuracy",
    save_best_only=True,
    verbose=1
)

cbs = [
    callbacks.EarlyStopping(
        monitor="val_accuracy",
        patience=15,
        restore_best_weights=True
    ),
    callbacks.ReduceLROnPlateau(
        monitor="val_loss",
        factor=0.5,
        patience=6,
        min_lr=1e-6
    ),
    checkpoint_cb
]

# =====
# 8) TRAIN
# =====
history = model.fit(
    X_train_aug, y_train_aug,
    validation_data=(X_val, y_val),
    epochs=EPOCHS,
    batch_size=BATCH_SIZE,
    class_weight=class_weight,
    callbacks=cbs,
    verbose=1
)

```

Hình 2.12: Train processing

Bước 4: Evaluation

Sau bước training thì đây cũng là một bước rất quan trọng và cần thiết. Ở bước Evaluation giúp chúng ta đánh giá được hiệu suất của mô hình trên tập dữ liệu kiểm tra

độc lập, nhằm đo lường khả năng tổng quát hóa của mô hình thông qua độ chính xác và hàm mất mát. Test_loss chỉ giá trị hàm mất mát và test_acc dùng để đánh giá độ chính xác. Loss càng thấp thì dự đoán càng tốt và ngược lại.

```
# 9) TEST
# =====
test_loss, test_acc = model.evaluate(X_test, y_test, verbose=0)
print(f"✅ TEST ACCURACY: {test_acc:.4f}")
```

Hình 2.13: Evaluation

Bước 5: Save model

Model được lưu dưới dạng cấu trúc SavedModel với Saved_model_best, đồng thời xóa các thư mục cũ để tránh nhầm lẫn hoặc delay. Trong cấu trúc này chứa kiến trúc, trọng số và graph.

```
# 10) EXPORT SAVEDMODEL (LAST + BEST)
# =====
# Clean old folders to avoid confusion
def safe_rmtree(path):
    if os.path.isdir(path):
        import shutil
        shutil.rmtree(path, ignore_errors=True)

safe_rmtree(LAST_SAVEDMODEL_DIR)
safe_rmtree(BEST_SAVEDMODEL_DIR)

print("📁 Export LAST SavedModel ->", LAST_SAVEDMODEL_DIR)
model.export(LAST_SAVEDMODEL_DIR)

print("📁 Export BEST SavedModel ->", BEST_SAVEDMODEL_DIR)
best_model = tf.keras.models.load_model(BEST_KERAS_PATH)
best_model.export(BEST_SAVEDMODEL_DIR)
```

Hình 2.14: Save model

2.6. Real-time inference

Mục đích: nhận diện gesture real-time từ webcam, tích hợp vào template với Gesture Voter.

Quy trình thực hiện:

Bước 1: Setup

a) Load model và metadata

```
# Model paths (dùng chung với toàn project):
# Nhóm17_DoAnXuLyAnhSo_HCMUTE/models/SavedModel/...
GESTURE_MODEL_PATH = os.path.join(project_root, "models", "SavedModel", "saved_model_best")
METADATA_PATH = os.path.join(project_root, "models", "SavedModel", "metadata.pkl")

# Load model
if not os.path.exists(GESTURE_MODEL_PATH):
    raise FileNotFoundError(f"Không tìm thấy model: {GESTURE_MODEL_PATH}")
if not os.path.exists(METADATA_PATH):
    raise FileNotFoundError(f"Không tìm thấy metadata: {METADATA_PATH}")
gesture_model = tf.saved_model.load(GESTURE_MODEL_PATH)

# Kiểm tra và lấy signature
if hasattr(gesture_model, 'signatures') and 'serving_default' in gesture_model.signatures:
    model_signature = gesture_model.signatures['serving_default']
    print(f" → Using signature: serving_default")
else:
    raise ValueError("SavedModel không có signature 'serving_default'!")

# Load metadata
with open(METADATA_PATH, 'rb') as f:
    metadata = pickle.load(f)

    if not isinstance(metadata, dict):
        raise ValueError(f"metadata.pkl không phải dict! Type: {type(metadata)}")
    # Kiểm tra 'labels'
    if 'labels' not in metadata:
        raise ValueError(f"metadata.pkl không có 'labels'!")
    labels_list = metadata['labels']
    if not isinstance(labels_list, list):
        raise ValueError(f"metadata['labels'] phải là list, nhận được: {type(labels_list)}")
    if len(labels_list) == 0:
        raise ValueError("metadata['labels'] không được rỗng!")
    # Tạo label_mapping: {0: label0, 1: label1, ...}
    label_mapping = {i: label for i, label in enumerate(labels_list)}
    # Extract symmetric gestures từ labels (các label bắt đầu bằng 'S_')
    SYMMETRIC_GESTURES = set([label for label in labels_list if label.startswith(SYMMETRIC_PREFIX)])
    num_classes = len(labels_list)
```

Hình 2.15: Load model và metadata

b) GestureVoter class

```

class GestureVoter:
    def __init__(self,
                  acceptance_time=GESTURE_VOTER_ACCEPTANCE_TIME,
                  vote_lifetime=GESTURE_VOTER_VOTE_LIFETIME,
                  vote_threshold=GESTURE_VOTER_VOTE_THRESHOLD,
                  min_votes=GESTURE_VOTER_MIN_VOTES):

        # Validation: acceptance_time phải ≤ vote_lifetime
        # Nếu không, vote đầu tiên sẽ bị xóa trước khi đạt acceptance_time
        if acceptance_time > vote_lifetime:
            raise ValueError(
                f"acceptance_time ({acceptance_time}s) phải ≤ vote_lifetime ({vote_lifetime}s). "
            )

        self.acceptance_time = acceptance_time
        self.vote_lifetime = vote_lifetime
        self.vote_threshold = vote_threshold
        self.min_votes = min_votes
        self.votes = deque()
        self.current_gesture = None
        self.current_confidence = 0.0

    def vote(self, prediction, confidence):
        current_time = time.time()

        # Kiểm tra nếu gesture thay đổi: reset votes và current_gesture
        # Điều này đảm bảo gesture mới bắt đầu từ đầu, không bị ảnh hưởng bởi votes cũ
        if self.current_gesture is not None and self.current_gesture != prediction:
            # Gesture thay đổi → reset hoàn toàn để bắt đầu gesture mới
            self.votes.clear()
            self.current_gesture = None
            self.current_confidence = 0.0

        # Thêm vote mới
        self.votes.append((current_time, prediction, confidence))

```

```

        # Xóa votes cũ (quá vote_lifetime)
        cutoff_time = current_time - self.vote_lifetime
        while self.votes and self.votes[0][0] < cutoff_time:
            self.votes.popleft()

        # ĐIỀU KIỆN 1: Đủ số votes đầu vào (min_votes)
        # Phải có ít nhất min_votes mới bắt đầu kiểm tra các điều kiện khác
        if len(self.votes) < self.min_votes:
            return None, 0.0

        # ĐIỀU KIỆN 2: Đủ thời gian giữ tay (acceptance_time)
        # Phải giữ tay trong ít nhất acceptance_time từ vote đầu tiên
        first_vote_time = self.votes[0][0]
        if (current_time - first_vote_time) < self.acceptance_time:
            return None, 0.0

        # ĐIỀU KIỆN 3: Đủ tỷ lệ đồng ý (vote_threshold)
        # Tính tỷ lệ dựa vào TẤT CẢ votes trong vote_lifetime (KHÔNG phải chỉ trong acceptance_time)
        # Sau khi reset, tất cả votes đều là gesture mới
        predictions = [pred for _, pred, _ in self.votes]
        counter = Counter(predictions)
        gesture, count = counter.most_common(1)[0]
        ratio = count / len(self.votes) # Tỷ lệ = số votes gesture / tổng số votes trong vote_lifetime

        # Chỉ chấp nhận nếu đạt vote_threshold (80%)
        if ratio >= self.vote_threshold:
            self.current_gesture = gesture
            self.current_confidence = ratio
            return gesture, ratio

        # Chưa đạt threshold: trả về None để hiển thị progress
        return None, 0.0

```

```

def get_progress(self):
    if len(self.votes) < self.min_votes:
        return 0.0, 0.0
    first_vote_time = self.votes[0][0]
    elapsed_time = time.time() - first_vote_time
    time_progress = min(100, (elapsed_time / self.acceptance_time) * 100)
    predictions = [pred for _, pred, _ in self.votes]
    counter = Counter(predictions)
    count = counter.most_common(1)[0][1]
    vote_progress = (count / len(self.votes)) * 100
    return vote_progress, time_progress

def reset(self):
    self.votes.clear()
    self.current_gesture = None
    self.current_confidence = 0.0

# Global gesture voters (một voter cho mỗi hand)
gesture_voters = {}

```

Hình 2.16: GestureVoter class

c) Hàm normalize features

```
def normalize_features(landmarks_array):
    landmarks = np.asarray(landmarks_array, dtype=np.float32)
    x_coords = landmarks[:, 0]
    y_coords = landmarks[:, 1]

    # Relative normalization về wrist (landmark 0)
    wrist_x, wrist_y = x_coords[0], y_coords[0]
    relative_x = x_coords - wrist_x
    relative_y = y_coords - wrist_y

    # Scale normalization
    max_value = max(np.max(np.abs(relative_x)), np.max(np.abs(relative_y)))
    if max_value > 0:
        normalized_x = relative_x / max_value
        normalized_y = relative_y / max_value
    else:
        normalized_x, normalized_y = relative_x, relative_y

    # Tính trung bình của 4 MCP joints (Index, Middle, Ring, Pinky)
    mcp_x_avg = np.mean(normalized_x[FINGER_MCP_INDICES])
    mcp_y_avg = np.mean(normalized_y[FINGER_MCP_INDICES])

    # Y_hand: vector từ wrist (0,0) đến mean(MCPs) - đây là hướng lên của bàn tay
    y_hand_x = mcp_x_avg # Wrist = (0,0) sau relative normalization
    y_hand_y = mcp_y_avg

    # X_hand: vector từ index_MCP đến pinky_MCP - đây là hướng ngang của bàn tay
    x_hand_x = normalized_x[PINKY_MCP_IDX] - normalized_x[INDEX_MCP_IDX]
    x_hand_y = normalized_y[PINKY_MCP_IDX] - normalized_y[INDEX_MCP_IDX]

    EPSILON = 1e-6 # Ngưỡng tối thiểu để tránh chia cho 0

    # Normalize Y_hand TRƯỚC (ưu tiên orientation) - đây là trục chính của bàn tay
    y_hand_mag = np.sqrt(y_hand_x**2 + y_hand_y**2)
    if y_hand_mag > EPSILON:
        y_hand_x_normalized = y_hand_x / y_hand_mag
        y_hand_y_normalized = y_hand_y / y_hand_mag
    else:
```

```
else:
    # Fallback: nếu không tính được X_hand (2 MCPs trùng nhau), dùng vector vuông góc với Y_hand
    # Đảm bảo X_hand vuông góc với Y_hand để tạo hệ trục chuẩn
    x_hand_x_normalized = -y_hand_y_normalized
    x_hand_y_normalized = y_hand_x_normalized
    # Normalize lại vector vuông góc
    x_hand_mag_fallback = np.sqrt(x_hand_x_normalized**2 + x_hand_y_normalized**2)
    if x_hand_mag_fallback > EPSILON:
        x_hand_x_normalized = x_hand_x_normalized / x_hand_mag_fallback
        x_hand_y_normalized = x_hand_y_normalized / x_hand_mag_fallback
    else:
        # Fallback cuối cùng: nếu vẫn không được, dùng (1, 0) mặc định
        x_hand_x_normalized = 1.0
        x_hand_y_normalized = 0.0

    # Flatten thành NUM_FEATURES features (42 landmarks + 2 Y_hand + 2 X_hand)
    feats = np.empty(NUM_FEATURES, dtype=np.float32)
    for i in range(NUM_LANDMARKS):
        feats[2*i] = float(normalized_x[i])
        feats[2*i+1] = float(normalized_y[i])

    # Thêm Y_hand features (indices 42, 43) - đã normalize và kiểm tra
    feats[NUM_LANDMARKS * 2] = float(y_hand_x_normalized) # Index 42: Y_hand X component
    feats[NUM_LANDMARKS * 2 + 1] = float(y_hand_y_normalized) # Index 43: Y_hand Y component

    # Thêm X_hand features (indices 44, 45) - đã normalize và kiểm tra
    feats[NUM_LANDMARKS * 2 + 2] = float(x_hand_x_normalized) # Index 44: X_hand X component
    feats[NUM_LANDMARKS * 2 + 3] = float(x_hand_y_normalized) # Index 45: X_hand Y component

    return feats
```

Hình 2.17: Hàm *normalize features*

d) Hàm map prediction label

```
479 # ----- 2.3. Map Prediction Label Function -----
480 def map_prediction_label(prediction, handedness_label, SYMMETRIC_GESTURES):
481     # Symmetric gestures: bỏ prefix "S_"
482     if prediction in SYMMETRIC_GESTURES:
483         return prediction.replace(SYMMETRIC_PREFIX, "") if prediction.startswith(SYMMETRIC_PREFIX) else prediction
484
485     # Asymmetric gestures: giữ nguyên prefix LH/RH để biết rõ tay trái/phải
486     if prediction.startswith(ASYMMETRIC_PREFIX_RH):
487         # Model predict "A_RH_FanLeft" → hiển thị "RH_FanLeft"
488         return prediction.replace(ASYMMETRIC_PREFIX_RH, "RH_")
489
490     elif prediction.startswith(ASYMMETRIC_PREFIX_LH):
491         # Model predict "A_LH_FanLeft" → hiển thị "LH_FanLeft"
492         return prediction.replace(ASYMMETRIC_PREFIX_LH, "LH_")
493
494     # Fallback
495     return prediction
```

Hình 2.18: Hàm map prediction label

Bước 2: Xử lý trong 'update_frame()'

a) Extract features from landmarks (sau EMA smoothing)

```
1826 orientation_valid, y_hand_mag, x_hand_mag = check_orientation_validity(landmarks_normalized)
1827 if not orientation_valid:
1828     # Orientation không ổn định → không đáng tin, bỏ qua prediction
1829     if hand_idx in gesture_voters:
1830         gesture_voters[hand_idx].reset() # Reset voter để tránh tích lũy votes sai
1831     gesture_display = f"Unstable Orientation (Y:{y_hand_mag:.3f}, X:{x_hand_mag:.3f})"
1832     vote_confidence = 0.0
1833     # Vẽ bounding box nhưng không hiển thị gesture
1834     # (sẽ được xử lý ở phần vẽ bên dưới)
1835 else:
1836     # Extract NUM_FEATURES features từ normalized landmarks [0,1] (42 landmarks + 2 Y_hand + 2 X_hand)
1837     features = normalize_features(landmarks_normalized).astype(np.float32)
```

Hình 2.19: Kiểm tra độ ổn định hướng bàn tay

b) Predict

```
1839 # Predict gesture
1840 features_resaped = features.reshape(1, NUM_FEATURES).astype(np.float32)
1841 # SavedModel đã được compiled khi save, không cần compile lại
1842 # tf.function compile function thành graph → nhanh hơn rất nhiều
1843 predictions = predict_gesture(features_resaped)
1844 # Convert TensorFlow tensor về numpy
1845 predictions_np = predictions.numpy() if hasattr(predictions, 'numpy') else np.array(predictions)
1846 probs = predictions_np[0] # Probability distribution
1847
1848 # Lấy top-1 prediction
1849 top1_idx = int(np.argmax(probs))
1850 confidence = float(probs[top1_idx])
1851 class_id = top1_idx
```

Hình 2.20: Dự đoán Gesture

c) GestureVoter – Adaptive voting

```
1880 if hand_idx not in gesture_voters:
1881     gesture_voters[hand_idx] = GestureVoter(
1882         acceptance_time=GESTURE_VOTER_ACCEPTANCE_TIME,
1883         vote_lifetime=GESTURE_VOTER_VOTE_LIFETIME,
1884         vote_threshold=GESTURE_VOTER_VOTE_THRESHOLD,
1885         min_votes=GESTURE_VOTER_MIN_VOTES
1886     )
```

Hình 2.21: Khởi tạo GestureVoter

d) Xử lý kết quả và map label

```

1898 # Xử lý kết quả và map label
1899 if final_label is None:
1900     # Chưa đạt threshold → hiển thị progress
1901     vote_progress, time_progress = gesture_voters[hand_idx].get_progress()
1902     gesture_display = f"Processing... ({vote_progress:.0f}% votes, {time_progress:.0f}% time)"
1903     vote_confidence = 0.0
1904 else:
1905     # Đã đạt threshold → validate handedness cho asymmetric gestures
1906     # (handedness_label_clean đã được lấy ở trên)
1907
1908     # FILTER: Validate handedness cho asymmetric gestures
1909     is_valid_handedness, expected_hand, mismatch_reason = validate_handedness_for_prediction(
1910         final_label, handedness_label_clean
1911     )

```

Hình 2.22: Hậu xử lý dự đoán

e) Hiển thị

```

1962 # Label với gesture classification
1963 label = f"ID:{hand_idx} {handedness_label} | Gesture: {gesture_display} ({vote_confidence:.1f}%)"
1964
1965 # Vẽ bounding box
1966 cv2.rectangle(annotated_frame, (min_x, min_y), (max_x, max_y), color, 2)
1967 (text_width, text_height), baseline = cv2.getTextSize(
1968     label, cv2.FONT_HERSHEY_SIMPLEX, 0.5, 1
1969 )
1970 # Đảm bảo text box không vẽ ra ngoài frame (y >= 0)
1971 text_y_start = max(0, min_y - text_height - baseline - 3)
1972 text_y_end = min_y
1973 cv2.rectangle(
1974     annotated_frame,
1975     (min_x, text_y_start),
1976     (min_x + text_width, text_y_end),
1977     color,
1978     -1,
1979 )

```

```

2002 ✓ if result and result.hand_landmarks:
2003     current_hand_indices = set(range(len(result.hand_landmarks)))
2004 ✓ for hand_idx in list(gesture_voters.keys()):
2005 ✓     if hand_idx not in current_hand_indices:
2006         gesture_voters[hand_idx].reset()
2007         del gesture_voters[hand_idx]

```

Hình 2.23: Hiển thị kết quả

CHƯƠNG 3: THỰC NGHIỆM VÀ KẾT QUẢ

3.1. Thực nghiệm

Vào các ngày 19/12, 20/12 và 21/12 nhóm đã liên tục kiểm thử sản phẩm ở nhiều địa điểm với các điều kiện ánh sáng khác nhau (Thư viện tầng hầm, khu A, phòng học tại nhà của nhóm trưởng, khu vực ngoài trời gần hội trường lớn). Các lần thực nghiệm đều lặp lại cùng một quy trình kiểm tra, đảm bảo không bỏ sót cử chỉ tay nào có trong đồ án:

- Camera nhận diện nhấn “Start” để khởi động hệ thống nhận diện, buzzer bíp 2 lần nếu nhận được nhấn “Start”, hệ thống sẽ không khởi động nếu là nhấn khác. Cử chỉ đại diện cho nhấn “Start” là năm ngón tay duỗi thẳng, lòng bàn tay hướng ra camera biểu diễn số năm. Để dễ dàng điều khiển, nhóm thống nhất tất cả cử chỉ đều đảm bảo chiều từ cổ tay để các ngón tay là hướng lên (trừ hai cử chỉ tắt đèn).
- Hệ thống nhận diện khởi động, khi người dùng thực hiện kí hiệu điều khiển, hệ thống sẽ nhận diện tay, phân loại nhấn cho tay đó. Có được nhấn, buzzer kêu bíp 1 lần và ESP32 thực hiện điều khiển thiết bị tương ứng như sau:
 - Nhấn “FanOff”: Tắt quạt.
Cử chỉ tay: năm ngón tay duỗi thẳng, mu bàn tay hướng ra camera.
 - Nhấn “FanLeft”: Servo trục ngang thực hiện quay theo chiều sang trái.
Cử chỉ tay: ngón cái quay sang trái, các ngón còn lại nắm lại.
 - Nhấn “FanRight”: Servo trục ngang thực hiện quay theo chiều sang phải.
Cử chỉ tay: ngón cái quay sang phải, các ngón tay còn lại nắm lại.
 - Nhấn “FanUp”: Servo trục dọc thực hiện quay theo chiều quay lên.
Cử chỉ tay: ngón cái hướng lên, các ngón khác nắm lại, tương tự cử chỉ “Like” hay dùng thường ngày.
 - Nhấn “FanDown”: Servo trục dọc thực hiện quay theo chiều quay xuống.
Cử chỉ tay: ngón cái hướng xuống dưới, các ngón khác nắm lại, tương tự cử chỉ “Dislike”.
 - Nhấn “FanSpeed1”: Module điều khiển động cơ DC (L298) điều chỉnh tốc độ quạt về mức thấp.

Cử chỉ tay: ngón trỏ chỉ lên trên, các ngón khác co (nhắm) lại, tương tự cách biểu diễn số 1.

- Nhãn “FanSpeed2”: Module điều khiển động cơ DC (L298) điều chỉnh tốc độ quạt về mức trung bình.

Cử chỉ tay: ngón trỏ và ngón giữa chỉ lên trên, các ngón khác co (nhắm) lại, tương tự cách biểu diễn số 2.

- Nhãn “FanSpeed3”: Module điều khiển động cơ DC (L298) điều chỉnh tốc độ quạt về mức cao.

Cử chỉ tay: ngón cái, ngón trỏ và ngón giữa hướng lên, hai ngón còn lại gập vào, biểu diễn số 3.

- Nhãn “Light1On”: Mở đèn tầng 1.

Cử chỉ tay: ngón trỏ và ngón cái tạo thành một góc xấp xỉ góc vuông, các ngón còn lại co vào.

- Nhãn “Light1Off”: Tắt đèn tầng.

Cử chỉ tay: tương tự như nhãn “Light1On” nhưng chiều từ cổ tay đến các ngón hướng xuống dưới.

- Nhãn “Light2On”: Mở đèn tầng 2.

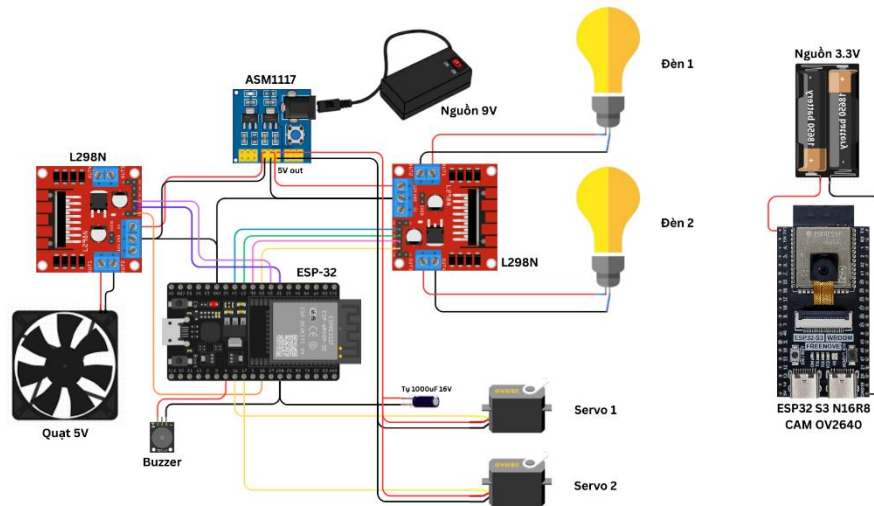
Cử chỉ tay: ngón giữa khép liền ngón trỏ tạo với ngón cái góc xấp xỉ góc vuông, các ngón còn lại co vào.

- Nhãn “Light2Off”: Tắt đèn tầng 2.

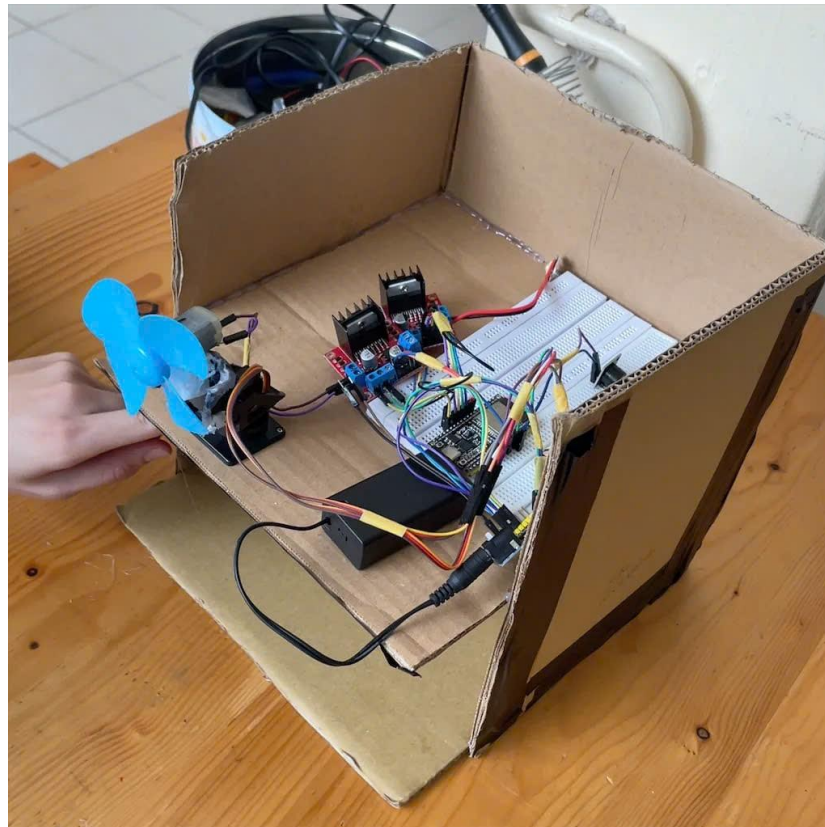
Cử chỉ tay: tương tự như nhãn “Light2On” nhưng chiều từ cổ tay đến các ngón hướng xuống dưới.

- Nhãn “Nothing”: Không nhận cử chỉ tay nào, chỉ có background.

Về phần cứng, luồng hoạt động cơ bản như sau: ESP32 có tích hợp camera ov2640 lấy ảnh từ cảm biến camera rồi gửi lên HTTP dưới dạng IP/stream lên router (wifi hotspot), PC là server lấy ảnh từ ESP32_Cam để xử lý. Sau khi có được tín hiệu, thực hiện gửi cho ESP32 khác (gọi là ESP32_Ctrl) đang hoạt động để điều khiển đèn và quạt.



Hình 3.1: Sơ đồ lắp mạch

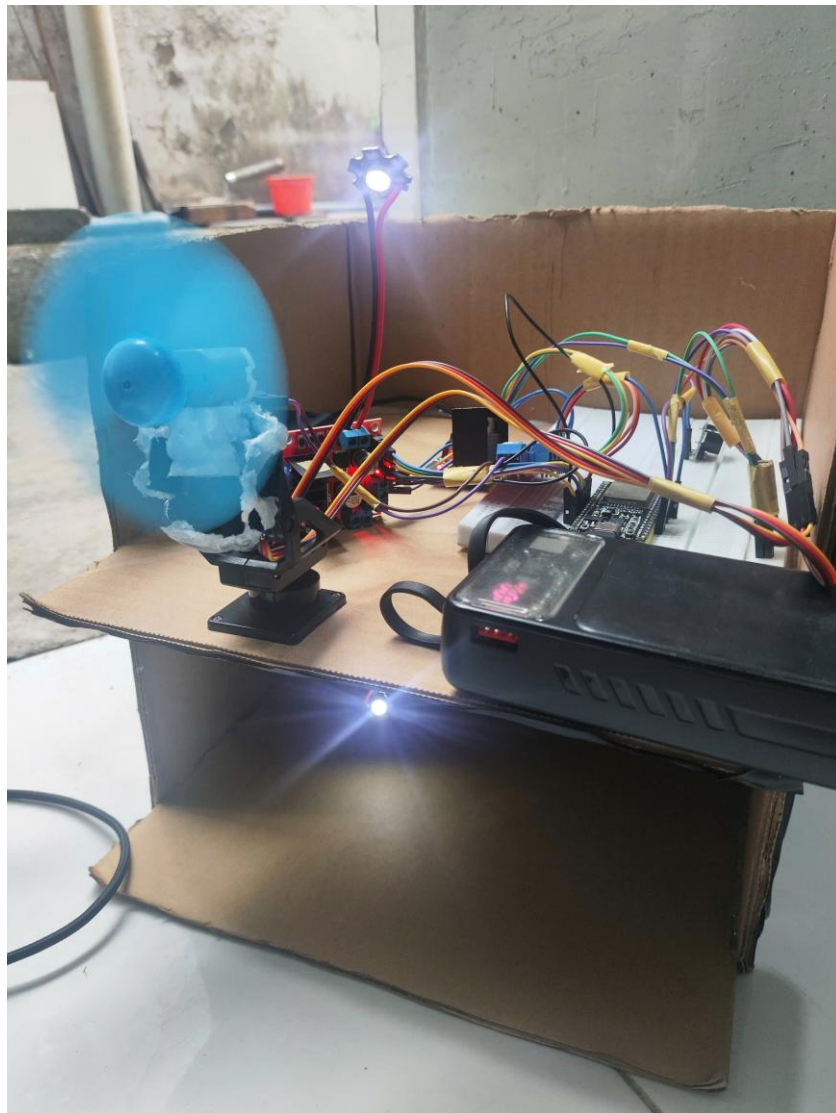


Hình 3.2: Quá trình xây dựng phần cứng

3.2. Kết quả thử nghiệm

Hệ thống hoạt động tốt trong môi trường ánh sáng ổn định, không bị quá chói hoặc quá tối. Hệ thống nhận diện thực hiện nhận diện và phân loại nhãn trong khoảng thời gian hợp lý (khoảng không quá 3 giây), hành động điều khiển được thực hiện ngay khi nhãn được xác định. Tuy nhiên, trong một số trường hợp, ảnh thu được có chất lượng kém hơn bình thường hoặc cử chỉ tay có góc bị quá lệch sẽ dẫn đến tình trạng

không nhận diện được, dẫn đến chậm trễ hoặc thậm chí nhầm sang tín hiệu khác nếu hai cử chỉ đó có các góc và chiều gần giống nhau.



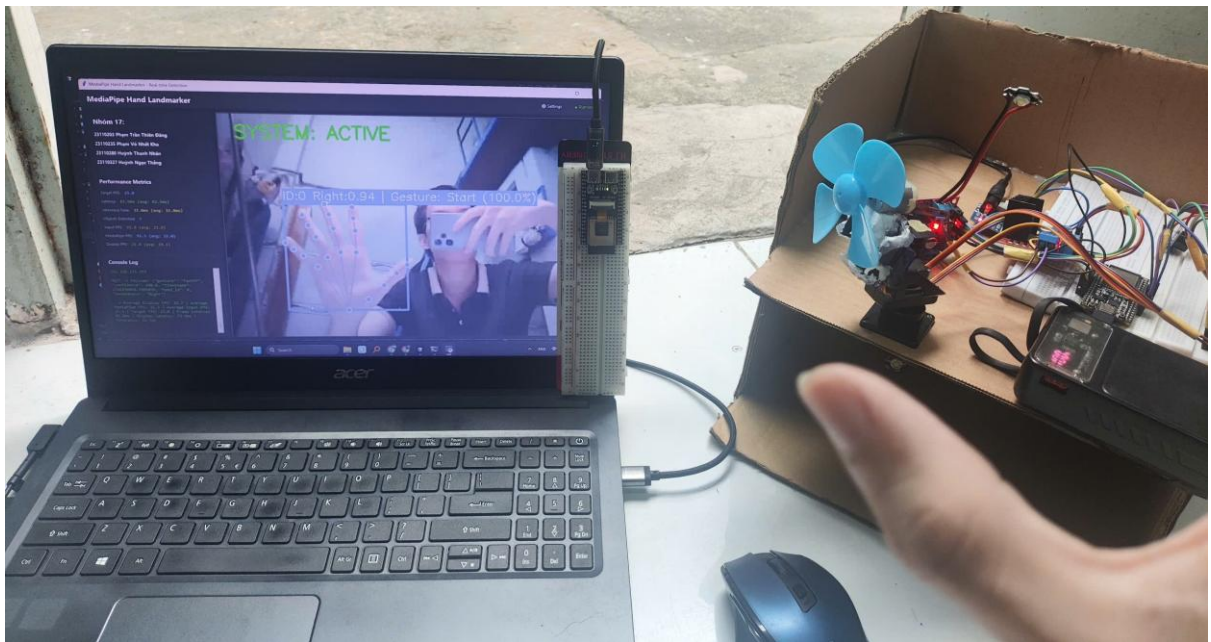
Hình 3.3: Hệ thống đang hoạt động

```
Console Log
-----
64.7ms | Display latency: 63.9ms |
Inference: 31.6ms

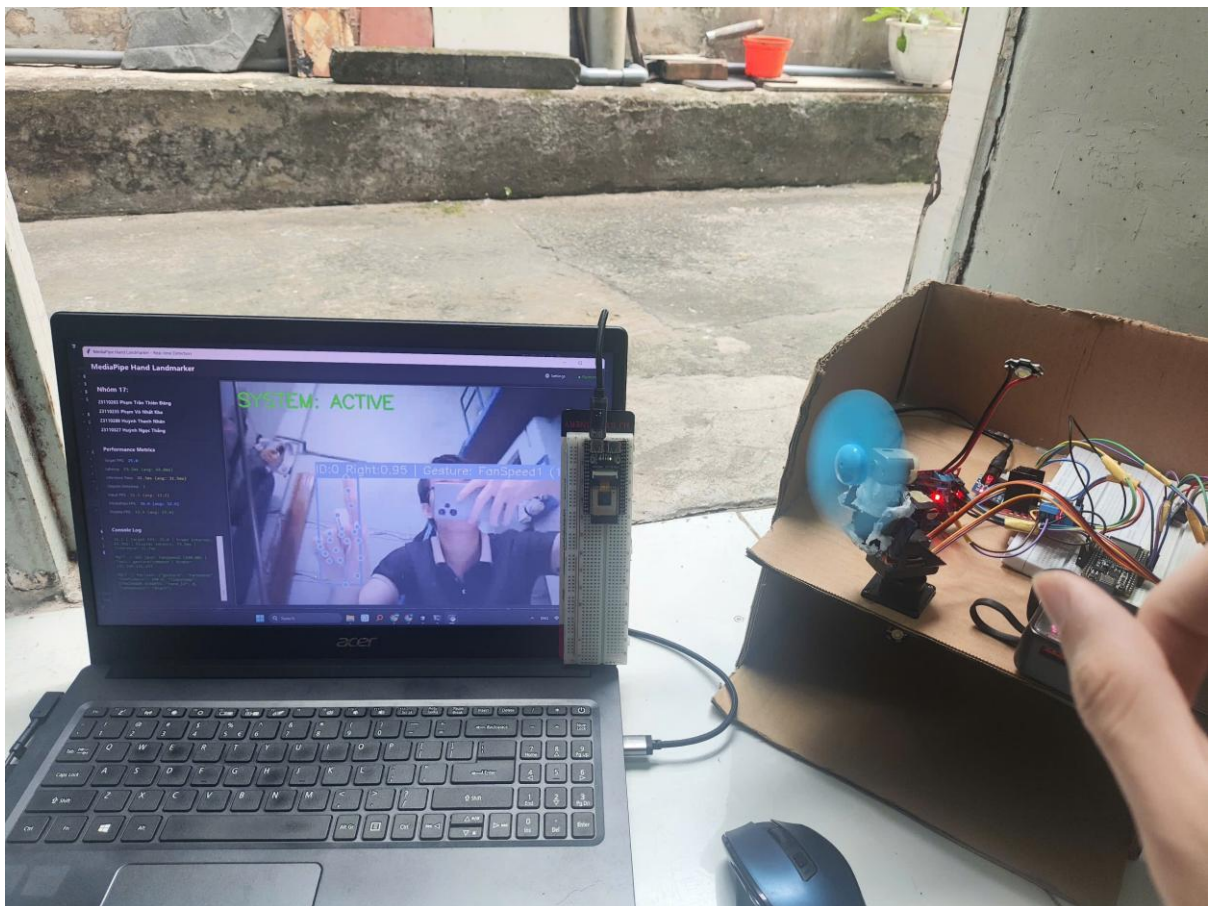
MQTT -> Gửi lệnh: Light2On (100.0%) |
Topic: gesture/command | Broker:
192.168.115.253

MQTT -> Payload: {"gesture": "Light2On",
"confidence": 100.0, "timestamp":
1766296101.9026961, "hand_id": 0,
"handedness": "Right"}
```

Hình 3.4: Ghi log trong quá trình chạy



Hình 3.5: Khởi động hệ thống bằng cử chỉ tay nhấn “Start”



Hình 3.6: Điều khiển quạt hoạt động ở tốc độ 1



Hình 3.7: Bật đèn tầng 1



Hình 3.8: Bật đèn tầng 2

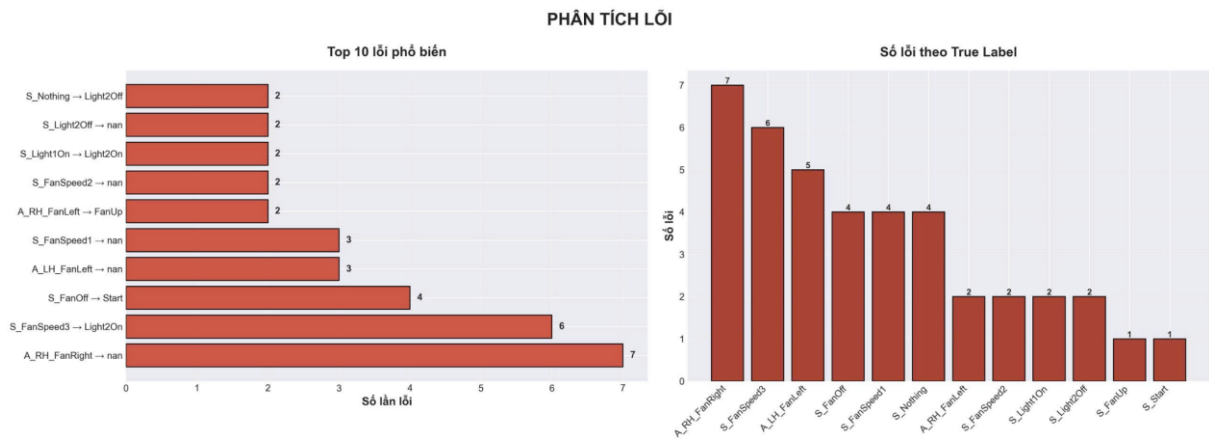


Hình 3.9: Tắt đèn tầng 1

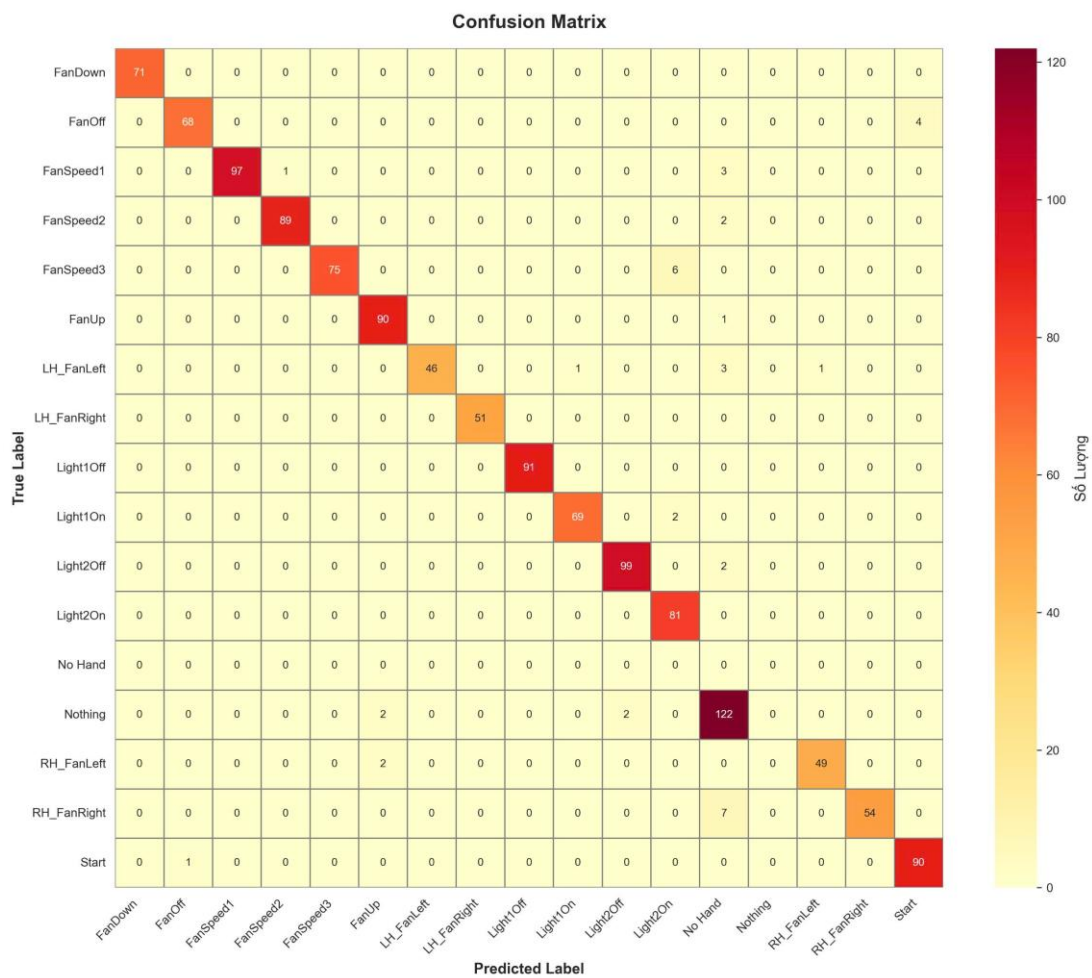


Hình 3.10: Xoay quạt

Cụ thể, sau khi kiểm thử với 1282 ảnh thực tế có link drive đính kèm trong folder, hệ thống nhận diện đúng 1242 ảnh, nhận diện sai 22 ảnh và không nhận diện được 18 ảnh, độ chính xác đạt 98.26% trên các ảnh nhận diện được tay (tính cả 18 ảnh không nhận diện được thì độ chính xác tổng thể là 96.9%).



Hình 3.11: Đồ thị thể hiện số lỗi nhận diện theo từng nhãn



Hình 3.12: Ma trận thống kê kết quả dự đoán theo từng nhãn



Hình 3.13: Đồ thị biểu diễn độ chính xác theo nhãn của hệ thống nhận diện

PHẦN KẾT LUẬN

1. Đánh giá kết quả

Hệ thống đang hoạt động tương đối ổn định, nhận diện được kí hiệu tay trong môi trường ánh sáng ổn định đúng như mục tiêu đề ra, tốc độ nhận diện cử chỉ tay nhanh, xử lý và trả kết quả thực thi hành động tốt, độ trễ tín hiệu thấp. Nhóm hoàn thành đồ án đúng thời gian cho phép.

1.1. Ưu điểm

Thiết kế và xây dựng thành công hệ thống bao gồm cả phần cứng, điều khiển cùng lúc quạt và hai đèn trong thời gian cho phép của đồ án.

Nhận diện cử chỉ tay, xử lý và thực hiện hành động từ xa.

Hoạt động real-time với độ trễ thấp.

Điều khiển các thiết bị bằng cả hai tay với nhiều góc độ.

Tương tác thân thiện với người dùng với âm thanh từ buzzer.

Xây dựng được các chương trình ứng dụng nhỏ, hỗ trợ việc thu thập dữ liệu, tiền xử lý diễn ra nhanh chóng hơn.

1.2. Nhược điểm

Do giới hạn về thiết bị thu nhận hình ảnh (chỉ xử lý 2D trên một camera) nên hình ảnh thu được có độ phân giải không quá cao và không nhận diện được hết chi tiết của bàn tay, đặc biệt thiếu sót chi tiết ở một số góc độ tay quá khuất.

Còn phụ thuộc vào tốc độ của đường truyền mạng.

2. Hướng phát triển

Sử dụng hai camera có độ phân giải cao hơn để nhận diện được nhiều góc độ và chi tiết bàn tay hơn, làm tăng độ chính xác khi nhận diện.

Thay vì dùng laptop như một server xử lý và phát tín hiệu, có thể dùng Raspberry Pi để giải quyết vấn đề đường truyền mạng, giúp hệ thống xử lý và trao đổi dữ liệu nhanh hơn.

Nâng cấp tập dữ liệu lớn hơn để điều khiển nhiều thiết bị hơn.

TÀI LIỆU THAM KHẢO

- [1] Ban biên tập TopDev, “OpenCV là gì? Học Computer Vision không khó!”, TopDev, truy cập vào lúc 22:26 ngày 16/12/2025 tại đường dẫn: <https://topdev.vn/blog/opencv-la-gi-hoc-computer-vision-khong-kho/>
- [2] Quang Trần, “MediaPipe: Live ML Solutions và ứng dụng vẽ bằng Hands Gestures”, Viblo, truy cập vào lúc 22:28 ngày 16/12/2025 tại đường dẫn: <https://viblo.asia/p/mediapipe-live-ml-solutions-va-ung-dung-ve-bang-hands-gestures-gAm5ymOV5db>
- [3] Ban biên tập TopDev, “TensorFlow là gì? Tìm hiểu về TensorFlow từ A đến Z”, TopDev, truy cập vào lúc 22:30 ngày 16/12/2025 tại đường dẫn: <https://topdev.vn/blog/tensorflow-la-gi/>
- [4] To Duc Thang, “Làm quen với Keras”, Viblo, truy cập vào lúc 22:32 ngày 16/12/2025 tại đường dẫn: <https://viblo.asia/p/lam-quen-voi-keras-gGJ59mxJ5X2>
- [5] CareerLink, “Pandas Là Gì? Tìm Hiểu Lợi ích Và Cách Hoạt động Của Pandas”, CareerLink, truy cập vào lúc 22:34 ngày 16/12/2025 <https://www.careerlink.vn/cam-nang-viec-lam/tu-van-nghe-nghiep/pandas-la-gi-tim-hieu-loi-ich-va-cach-hoat-dong-cua-pandas>
- [6] “Hướng dẫn phát hiện điểm mốc trên tay”, Google AI for Developers truy cập vào lúc 22:37 ngày 16/12/2025 tại đường dẫn: https://ai.google.dev/edge/mediapipe/solutions/vision/hand_landmarker?hl=vi
- [7] “On-Device, Real-Time Hand Tracking with MediaPipe”, truy cập vào lúc 22:40 ngày 16/12/2025 tại đường dẫn: <https://research.google/blog/on-device-real-time-hand-tracking-with-mediapipe/>

BẢNG KHỐI LƯỢNG CÔNG VIỆC

Họ và tên	Công việc chủ yếu
Phạm Trần Thiên Đăng	Thu thập dữ liệu và trích xuất đặc trưng
Phạm Võ Nhất Kha	Thiết kế và huấn luyện mô hình
Huỳnh Thanh Nhân	Tích hợp real-time và xây dựng giao diện
Huỳnh Ngọc Thắng	Lên ý tưởng chính cho đồ án, xây dựng phần cứng